

NanoPi M5

From FriendlyELEC WiKi

[查看中文](#)

Contents

- 1 Introduction
- 2 Hardware Spec
- 3 Diagram, Layout and Dimension
 - 3.1 Layout
- 4 Get Started
 - 4.1 Essentials You Need
 - 4.2 TF Cards We Tested
 - 4.3 Configure parameters for serial port
 - 4.4 Install OS
 - 4.4.1 Downloads
 - 4.4.1.1 Official image
 - 4.4.1.2 Tools (optional)
 - 4.4.2 Flashing the OS to the microSD card
 - 4.4.3 Install OS to UFS
 - 4.4.3.1 Option 1: Install OS via TF Card
 - 4.4.3.1.1 Flash Official OS to UFS
 - 4.4.3.1.2 Flash third party OS (Image file) to UFS
 - 4.4.3.2 Option 2: Install OS on Web Page
 - 4.4.3.3 Option 3: Install OS via USB
 - 4.4.3.3.1 Step 1: Download the required files and install the USB driver
 - 4.4.3.3.2 Step 2: Connect NanoPi-M5 to PC and Enter Installation Mode
 - 4.4.3.3.3 Step 3: Launch RKDevTool
 - 4.4.3.3.4 Step 4: Erase UFS
 - 4.4.3.3.5 Step 5: Begin Flashing
 - 4.4.4 Installing the System to M.2 or USB Drive
- 5 Work with FriendlyWrt
 - 5.1 Introduction to FriendlyWrt
 - 5.2 First boot
 - 5.3 Account & Password
 - 5.4 Login FriendlyWrt
 - 5.5 Recommended security settings
 - 5.6 Change LAN IP in LuCI
 - 5.7 Safe shutdown operation
 - 5.8 Soft Factory Reset
 - 5.9 Install Software Packages
 - 5.9.1 Package repositories
 - 5.9.2 Update Package List
 - 5.9.3 List Available Packages
 - 5.9.4 List Installed Packages
 - 5.9.5 Install Packages
 - 5.9.6 Remove Packages
 - 5.10 Configure the function of the user button
 - 5.11 Some common issues of FriendlyWrt
 - 5.12 Use USB2LCD to view IP and temperature
 - 5.13 How to use USB WiFi
 - 5.13.1 Check USB WiFi Device with Command Line Utility
 - 5.13.2 Configure a USB WiFi Device as AP
 - 5.13.3 Common USB WiFi issues
 - 5.13.4 Change the default WiFi hotspot configuration
 - 5.14 Work with Docker Applications
 - 5.14.1 Work with Docker: Install JellyFin
 - 5.14.2 Work with Docker: Install Personal Nextcloud
 - 5.14.3 Docker FAQ and solutions
 - 5.14.3.1 Unable to access the network services provided by the Docker container
 - 5.15 Mount smbfs
 - 5.16 Use sdk to compile the package
 - 5.16.1 Install the compilation environment
 - 5.16.2 Download and decompress sdk from the network disk
 - 5.16.3 Compile the package
 - 5.16.4 Install the apk to NanoPi
 - 5.17 Build FriendlyWrt using GitHub Actions
- 6 Work with FriendlyCore
 - 6.1 FriendlyCore User Account
 - 6.2 Update Software Packages
 - 6.3 Setup Network Configurations
 - 6.3.1 Set static IP address
 - 6.3.2 Set a DNS
 - 6.3.3 Set up to use another network interface
 - 6.4 Setup Wi-Fi

- 6.5 Install the kernel-header package
 - 6.6 Delete Qt5 and related files
- 7 Work with Android
 - 7.1 WiFi models supported by Android
 - 7.1.1 M.2 WiFi Module
 - 7.1.1.1 Usb Dongle
 - 7.2 Bluetooth models supported by Android
 - 7.2.1 Bluetooth Adapters
 - 7.2.2 Bluetooth Remote
 - 7.3 How to use ADB
 - 7.3.1 USB connection
 - 7.3.2 For Android Tablet
 - 7.3.3 For Android TV
 - 7.3.4 Using ADB
 - 7.3.5 Using ADB over the network
 - 7.4 How to Change Default Launcher in Android TV
 - 7.5 Wired networks on Android
- 8 Work with Debian11 Desktop
 - 8.1 Introduction to Debian11 Desktop
 - 8.2 Account & Password
 - 8.3 View IP address
 - 8.4 Connect to Debian via SSH
 - 8.5 Update Software Packages
 - 8.6 Install x11vnc Server on Debian for Remote Access
 - 8.6.1 Install x11vnc server
 - 8.6.2 Set your password
 - 8.6.3 Setup x11vnc server with systemd auto start up
 - 8.6.4 Testing remote access
 - 8.7 Install the kernel-header package
 - 8.8 Change time zone
 - 8.8.1 Check the current time zone
 - 8.8.2 List all available time zones
 - 8.8.3 Set the time zone (e.g. Shanghai)
 - 8.9 Change startup LOGO and Wallpaper
 - 8.9.1 Change startup LOGO
 - 8.9.1.1 Approach 1: On a Linux PC
 - 8.9.1.2 Approach 2: No PC — with eFlasher
 - 8.9.2 Change Wallpaper
 - 8.10 Soft Factory Reset
 - 8.11 Start the program automatically at startup(For example Kodi)
 - 8.12 Disable auto-mounting
 - 8.13 Setup Chinese language and Input method
 - 8.13.1 Setup Chinese language
 - 8.13.2 Installing Chinese input method
 - 8.14 Installing Plex Multimedia Server
 - 8.15 Install Docker on Debian
 - 8.16 How to test NPU
 - 8.17 How to test VPU
 - 8.18 WiFi Connection
 - 8.18.1 Gui
 - 8.18.2 Console
 - 8.19 Cancel auto-login
 - 8.20 Test OpenGL ES
 - 8.21 HDMI/DP LCD Resolution
 - 8.22 HiDPI and display scaling
 - 8.23 Adjust HDMI overscan
 - 8.23.1 Query which resolutions the display supports
 - 8.23.2 Set resolution
 - 8.23.3 Adjust the HDMI overscan
 - 8.23.4 Automatic adjustment at boot
 - 8.24 Chromium web browser
 - 8.24.1 GPU
 - 8.24.2 VPU
 - 8.25 Test hardware encoding
 - 8.25.1 Check Supported Hardware Decoding Formats
- 9 Work with Debian10 Desktop
- 10 Buildroot Linux
- 11 How to Compile
 - 11.1 Setup Development Environment
 - 11.1.1 Method 1: Using docker to cross-compile
 - 11.1.2 Method 2: Setup build environment on the host machine
 - 11.1.2.1 Install required packages
 - 11.1.2.2 Setting the compiler path
 - 11.2 Build Openwrt/Friendlywrt
 - 11.2.1 Download FriendlyWrt 25.12 source code
 - 11.2.2 First compilation step
 - 11.2.3 Secondary compilation steps
 - 11.2.4 Build u-boot only
 - 11.2.5 Build kernel only
 - 11.2.6 Build friendlywrt only
 - 11.3 Build Buildroot

- 11.4 Build Other Linux
 - 11.4.1 Kernel and u-boot versions
 - 11.4.2 Build kernel linux-v6.1.y
 - 11.4.3 Build u-boot v2017.09
 - 11.4.4 Running the build
 - 11.4.4.1 Install to target board
 - 11.4.4.2 Packaging and creating an SD image
 - 11.4.4.3 USB flashing
 - 11.4.4.3.1 Linux
- 11.5 Build the code using scripts
 - 11.5.1 Download scripts and image files
 - 11.5.2 Compile the kernel
 - 11.5.3 Compile the kernel headers
 - 11.5.4 Compile the uboot
 - 11.5.5 Generate new image
- 11.6 Building AOSP from source
 - 11.6.1 Hardware and Software Requirements
 - 11.6.2 Download source from the netdrive
 - 11.6.3 Tablet profile build (First Build)
 - 11.6.4 TV profile build (First Build)
 - 11.6.5 Second build
 - 11.6.6 Running your AOSP build
 - 11.6.6.1 USB Flashing
 - 11.6.6.2 SD-to-eMMC Flashing
 - 11.6.7 Pack the new SD Image
- 12 Using On-Board Hardware Resources
 - 12.1 VPU
 - 12.2 NPU
 - 12.3 DTS files
 - 12.4 GPIO
 - 12.5 Serial Port
 - 12.6 RTC
 - 12.7 Watch dog
- 13 Usage of RKLLM and Large Language Model Testing
 - 13.1 Demo: Model Inference in a Docker Container
 - 13.2 Switching Large Language Models
 - 13.3 Currently Supported Models
 - 13.4 How to Convert a Model
- 14 Make a master board: clone your system and apps onto other boards
 - 14.1 Approach 1: build it on a Linux PC (sd-fuse)
 - 14.1.1 Backup rootfs
 - 14.1.2 Making a bootable image from a root filesystem
 - 14.2 Approach 2: no PC — build it directly with eFlasher
 - 14.2.1 Bit-by-bit copy (whole-drive, Ghost-style backup)
 - 14.2.2 File-based reverse extraction (recommended for master images)
- 15 Common Linux-based operating system operations
 - 15.1 Using ADB on Linux Systems
 - 15.1.1 Enabling ADB in Buildroot System
 - 15.1.2 Enabling ADB in Ubuntu and Debian Systems
 - 15.1.3 How to Connect
 - 15.2 Install Kernel Headers
 - 15.3 Setting Kernel Boot Parameters
 - 15.3.1 When the system needs to be installed on eMMC/UFS
 - 15.3.2 Change kernel boot parameters by compiling the kernel
 - 15.4 View the current kernel configuration of the running Linux system
 - 15.5 PWM Fan Control
 - 15.6 Clear the Automatically Generated MAC Address
 - 15.7 Scheduled Power On/Off
- 16 Unbricking Method
 - 16.1 Preparation
 - 16.2 Windows Users
 - 16.2.1 Download Required Files
 - 16.2.2 Enter Maskrom Mode to Erase the Storage Device
 - 16.3 Linux Users
 - 16.3.1 Download the Required Files
 - 16.3.2 Installation for upgrade_tool
 - 16.3.3 Enter Maskrom Mode to Erase the Storage Device
 - 16.4 Mac Users
- 17 How to use CAM415 MIPI Camera
- 18 Link to Rockchip Resources
- 19 Schematic, PCB CAD File
- 20 Update Logs
 - 20.1 2026-06-03
 - 20.1.1 eFlasher
 - 20.1.2 FriendlyWrt
 - 20.2 2026-04-21
 - 20.2.1 FriendlyWrt
 - 20.2.2 Ramdisk
 - 20.2.3 Debian 13(Trixie) GNOME Wayland Desktop
 - 20.3 2026-03-02
 - 20.3.1 Debian 13(Trixie) GNOME Desktop

- 20.3.2 Debian11 & Ubuntu20
- 20.4 2026-02-08
- 20.5 2026-01-12
 - 20.5.1 OpenMediaVault
 - 20.5.2 Debian Core
- 20.6 2025-12-22
 - 20.6.1 Android 14 / TV
 - 20.6.2 Linux
 - 20.6.3 FriendlyWrt
- 20.7 2025-10-14
 - 20.7.1 Android 14/TV
- 20.8 2025-10-10
 - 20.8.1 Android 14 / TV
 - 20.8.2 Eflasher
- 20.9 2025-09-05
 - 20.9.1 Android 14/TV
- 20.10 2025-08-11
 - 20.10.1 Ubuntu / Debian / FriendlyCore
- 20.11 2025-08-07
 - 20.11.1 Linux
 - 20.11.2 Android 14
- 20.12 2025-07-08
 - 20.12.1 FriendlyWrt
- 20.13 2025-06-30
 - 20.13.1 FriendlyWrt
- 20.14 2025-06-13
 - 20.14.1 Linux Kernel 6.1.y
- 20.15 2025-04-08

1 Introduction

The NanoPi M5 (as "M5") is an open-sourced mini SBC device, designed and developed by FriendlyElec. It is based on Rockchip RK3576 SoC, with 32-bits LPDDR4X/LPDDR5 RAM and UFS 2.0 storage(optional). It supports booting from TF cards and is compatible with a wide range of operating systems, including Android 14, Debian, Ubuntu, FriendlyWrt, Buildroot, OpenMediaVault, and more. It is also fully compatible with headless systems.

The RK3576 is an 8-core 64-bit processor featuring a powerful architecture with 4× ARM Cortex-A72 cores and 4× ARM Cortex-A53 cores. It is equipped with an ARM Mali G52 MC3 GPU and a 6 TOPS NPU, ensuring excellent graphics performance and AI processing capabilities. The RK3576 supports advanced video decoding and encoding, including 8K at 30fps and 4K at 120fps decoding (H.265/HEVC, VP9, AVS2, and AV1), as well as 4K at 60fps decoding (H.264/AVC) and 4K at 60fps encoding (H.265/HEVC and H.264/AVC). This makes it an ideal solution for high-performance multimedia and AI applications.

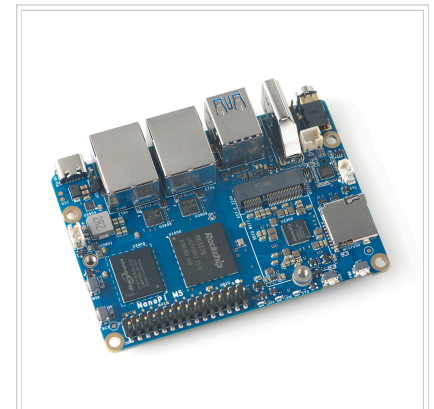
The NanoPi M5 offers rich hardware resources in a compact 90 x 62 mm form factor, featuring a wide range of interfaces, including M.2 NVMe port, USB 3.0 ports, MIPI-CSI, MIPI-DSI, an M.2 SDIO socket and a 30-pin header is provided to expose hardware resources for secondary development, including interfaces such as GPIO, I2C, I2S, PWM and SPI. It also has one HDMI port.

The NanoPi M5 supports USB type-C power delivery. FriendlyElec has released a carefully-designed custom CNC housing for it.

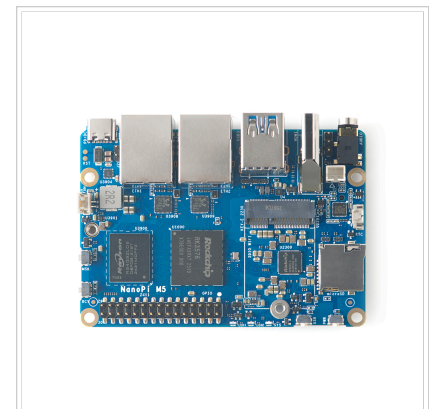
All in all, the NanoPi M5 is a versatile board featuring multiple Ethernet ports, light NAS capabilities, and video playback. It is an excellent choice for deploying AI applications locally, supporting inference for large AI models. Additionally, it supports Docker applications, making it an ideal platform for developers working on advanced solutions.

2 Hardware Spec

- SoC: Rockchip RK3576
 - CPU: Octa-core ARM processor with 4x Cortex-A72 cores and 4x Cortex-A53 cores
 - GPU: ARM Mali-G52 MC3, supports OpenGL ES 1.1, 2.0, 3.2, Vulkan 1.2, and OpenCL 2.0
 - VPU: 8K@30fps H.265/VP9 video decoder, 4K@60fps video encoder
 - NPU: 6 TOPS* @INT8 supports INT4, INT8, INT16, FP16, BF16, TF32 mixed operation
- RAM: 4GB LPDDR4X or 8GB/16GB LPDDR5
- Flash:
 - UFS 2.0 (optional)
 - 16MB SPI Nor Flash
- Network:
 - 2x 1Gbps Ethernet
 - Optional M.2 SDIO Wi-Fi
- USB: 2x USB 3.2 Gen 1 Type-A ports
- M.2 Connectors
 - M.2 M-Key connector with PCIe 2.1 x1 for SSDs
 - M.2 E-KEY SDIO connector
- microSD: support UHS-I
- Video input:
 - 2x 4-Lane MIPI-CSI V1.2
- Video output:



Overview



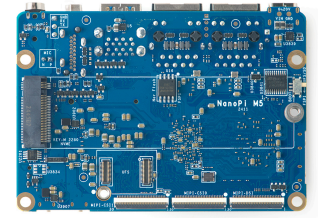
Front

- 1x Standard HDMI output ports
- support HDMI1.4 and HDMI2.0 operation
- support up to 10 bits Deep Color modes
- support up to 1080p@120Hz and 4096x2304@60Hz
- support 3-D video formats
- 1x 4-Lane MIPI-DSI, compatible with MIPI DPHY V2.0 or C-PHY V1.1
- Audio:
 - 1x 3.5mm jack for stereo headphone output
 - 1x 2.0mm PH-2A connector for analog microphone input
- GPIO:
 - 30-Pin 2.54mm header connector
 - up to 2x SPIs, 4x UARTs, 3x I2Cs, 1x SPDIFs, 5x PWMs, 20x GPIOs
- Debug: one Debug UART, 3-Pin 2.54mm header, 3.3V level, 1500000bps
- LEDs: 3x GPIO Controlled LEDs (SYS, LED1, LED2)
- others:
 - 2 Pin 1.27/1.25mm RTC battery input connector for low power RTC IC HYM8563TS
 - Buttons: MASK, User, Recovery, Power
 - Boot Mode Switch
 - Power-On Mode Jumper
 - 5V Fan connector
- Power supply: USB-C, support PD and DC, 6V~20V input
- PCB: 8-Layer ENIG, 90x62x1.6 mm
- Ambient Operating Temperature: 0°C to 70°C

3 Diagram, Layout and Dimension

3.1 Layout

- NanoPi M5 with LPDDR5



Back



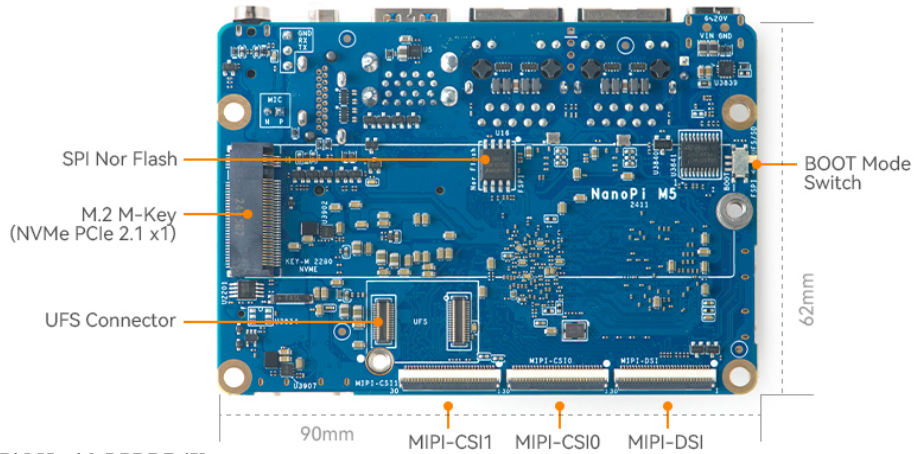
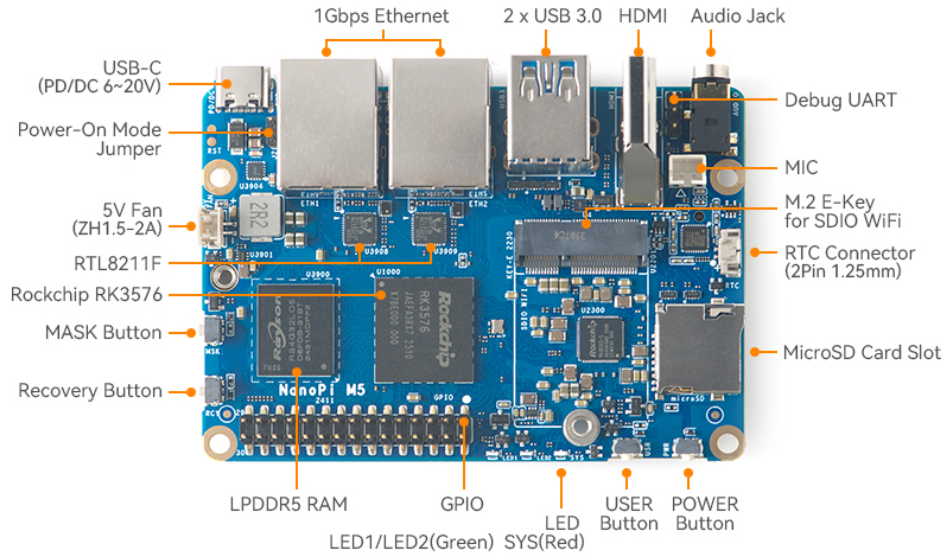
Case



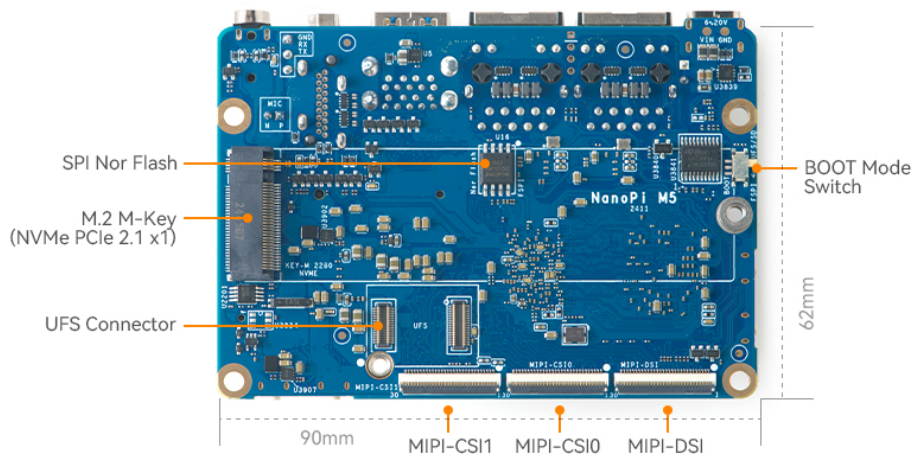
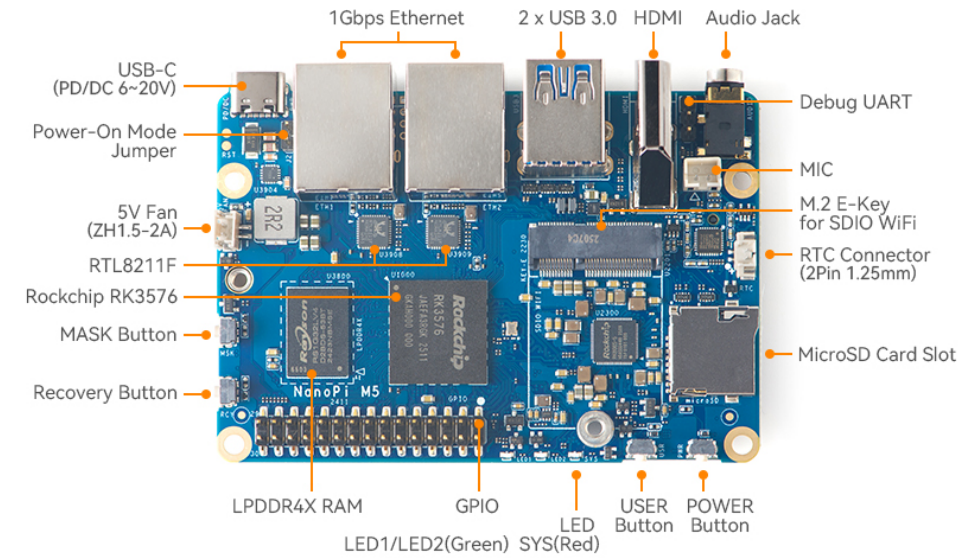
Case



Case



■ NanoPi M5 with LPDDR4X



■ Debug UART Pin Spec

3.3V level signals, 1500000bps

Pin#	Assignment	Description
1	GND	0V
2	UART0_TX_M0_DEBUG	output
3	UART0_RX_M0_DEBUG	input

■ M.2 Key-E

It has a SDIO and an UART, supports SDIO wifi.

■ 30-pin GPIO

Pin#	Assignment	Pin#	Assignment
1	VCC_3V3_S3*	2	VCC5V0_SYS_S5*
3	GPIO0_C2_I2C0_SDA	4	VCC5V0_SYS_S5*
5	GPIO0_C1_I2C0_SCL	6	GND
7	GPIO4_A5	8	GPIO4_B5_UART2_TX_I2C3_SCL
9	GND	10	GPIO4_B4_UART2_RX_I2C3_SDA
11	GPIO2_A6_UART8_TX	12	GPIO0_B5_PWM1_CH1_M0
13	GPIO2_A7_UART8_RX	14	GND
15	GPIO4_A2_PWM2_CH5	16	GPIO3_A0_UART3_TX_I2C7_SCL
17	VCC_3V3_S3*	18	GPIO3_A1_UART3_RX_I2C7_SDA
19	GPIO4_B1_SPI4_MOSI	20	GND
21	GPIO4_B2_SPI4_MISO	22	GPIO0_C4_PWM0_CH0_M0
23	GPIO4_B0_SPI4_CLK	24	GPIO4_B3_SPI4_CSN0
25	GND	26	GPIO4_A6_SPI3_MISO_UART6_RX
27	GPIO4_A7_SPI3_CLK	28	GPIO4_A4_SPI3_MOSI_UART6_TX
29	GPIO4_A3_SPI3_CSN0	30	GND

The logic level of all signals is 3.3V.

VCC_3V3_S3 is 3.3V output with a maximum output of 500mA.

VCC5V0_SYS_S5 is 5V output with a maximum output of 1A.

■ MIPI-DSI

0.5mm FPC Connector

Pin#	Assignment	Description
1,2,3	VCC5V0_SYS_S5	5V Power output
4,7,9,11,15,18,21,24,27,30	GND	Power and Signal Ground
5	I2C8_SDA_M2	3.3V, I2C Data, pulled up to 3.3V with 2.2K on M5
6	I2C8_SCL_M2	3.3V, I2C Clock, pulled up to 3.3V with 2.2K on M5
8	GPIO0_D0	3.3V, GPIO
10	PWM1_CH0_M0	3.3V, GPIO/PWM
12	GPIO0_C6	3.3V, GPIO
13	/NC	No Connection
14	GPIO0_C5	3.3V, GPIO
16	MIPI_DPHY_DSI_TX_D3N	MIPI TX Lane3 output N
17	MIPI_DPHY_DSI_TX_D3P	MIPI TX Lane3 output P
19	MIPI_DPHY_DSI_TX_D2N	MIPI TX Lane2 output N
20	MIPI_DPHY_DSI_TX_D2P	MIPI TX Lane2 output P
22	MIPI_DPHY_DSI_TX_D1N	MIPI TX Lane1 output N
23	MIPI_DPHY_DSI_TX_D1P	MIPI TX Lane1 output P
25	MIPI_DPHY_DSI_TX_D0N	MIPI TX Lane0 output N
26	MIPI_DPHY_DSI_TX_D0P	MIPI TX Lane0 output P
28	MIPI_DPHY_DSI_TX_CLKN	MIPI TX Clock output N
29	MIPI_DPHY_DSI_TX_CLKP	MIPI TX Clock output P

■ MIPI-CSI

0.5mm FPC Connector

Pin#	MIPI-CSI0	MIPI-CSI1	Description
1,2	VCC5V0_SYS_S5	VCC5V0_SYS_S5	5V Power output
3,13,15,18,21,24,27,30	GND	GND	Power and Signal Ground
4	MIPI_DPHY_CSI2_RX_CLKP	MIPI_DPHY_CSI4_RX_CLKP	MIPI RX Clock input
5	MIPI_DPHY_CSI2_RX_CLKN	MIPI_DPHY_CSI4_RX_CLKN	MIPI RX Clock input
6	VCC_1V8_S3	VCC_1V8_S3	1.8V Power output, 100mA Max
7	/NC	/NC	No Connection
8	CAM_SYNC	CAM_SYNC	Have been connected together or for sensor synchronization
9	I2C4_SCL_M3	I2C5_SCL_M3	1.8V, I2C Clock, pulled up to 1.8V with 2.2K on M5
10	I2C4_SDA_M3	I2C5_SDA_M3	1.8V, I2C pulled up to 1.8V with 2.2K on M5
11	GPIO3_C5	GPIO3_A4	1.8V, GPIO
12	GPIO3_C6	GPIO4_C7	1.8V, GPIO
14	CAM_CLK1_OUT_M0	CAM_CLK2_OUT_M0	1.8V, Clock output for Sensor
16	MIPI_DPHY_CSI1_RX_D3P/MIPI_DPHY_CSI2_RX_D1P	MIPI_DPHY_CSI3_RX_D3P/MIPI_DPHY_CSI4_RX_D1P	MIPI RX Lane3 input
17	MIPI_DPHY_CSI1_RX_D3N/MIPI_DPHY_CSI2_RX_D1N	MIPI_DPHY_CSI3_RX_D3N/MIPI_DPHY_CSI4_RX_D1N	MIPI RX Lane3 input
19	MIPI_DPHY_CSI1_RX_D2P/MIPI_DPHY_CSI2_RX_D0P	MIPI_DPHY_CSI3_RX_D2P/MIPI_DPHY_CSI4_RX_D0P	MIPI RX Lane2 input
20	MIPI_DPHY_CSI1_RX_D2N/MIPI_DPHY_CSI2_RX_D0N	MIPI_DPHY_CSI3_RX_D2N/MIPI_DPHY_CSI4_RX_D0N	MIPI RX Lane2 input
22	MIPI_DPHY_CSI1_RX_D1P	MIPI_DPHY_CSI3_RX_D1P	MIPI RX Lane1 input
23	MIPI_DPHY_CSI1_RX_D1N	MIPI_DPHY_CSI3_RX_D1N	MIPI RX Lane1 input
25	MIPI_DPHY_CSI1_RX_CLKP	MIPI_DPHY_CSI3_RX_CLKP	MIPI RX Clock input
26	MIPI_DPHY_CSI1_RX_CLKN	MIPI_DPHY_CSI3_RX_CLKN	MIPI RX Clock input
28	MIPI_DPHY_CSI1_RX_D0P	MIPI_DPHY_CSI3_RX_D0P	MIPI RX Lane0 input
29	MIPI_DPHY_CSI1_RX_D0N	MIPI_DPHY_CSI3_RX_D0N	MIPI RX Lane0 input

■ USB Port

Each USB 3.2 Gen 1 port has 1.5A overcurrent protection.

■ RTC

RTC backup current is 0.25µA TYP (VDD =3.0V, TA =25°C).
Connector P/N: Molex 53398-0271

4 Get Started

4.1 Essentials You Need

Before starting to use your NanoPi-M5 get the following items ready

- NanoPi-M5
- MicroSD Card/TF Card: Class 10 or Above, minimum 8GB SDHC
- USB C PD Charger (15W & above)

- If you need to develop and compile,you need a computer that can connect to the Internet. It is recommended to install Ubuntu 20.04 64-bit system and use the following script to initialize the development environment, or use docker container:
 - How to setup the Compiling Environment on Ubuntu bionic (<https://github.com/friendlyarm/build-env-on-ubuntu-bionic>)
 - docker-cross-compiler-novnc (<https://github.com/friendlyarm/docker-cross-compiler-novnc>)

4.2 TF Cards We Tested

Refer to: TFCardsWeTested

4.3 Configure parameters for serial port

Use the following serial parameters:

Baud rate	1500000
Data bit	8
Parity check	None
Stop bit	1
Flow control	None

4.4 Install OS

4.4.1 Downloads

4.4.1.1 Official image

Visit download link (<http://download.friendlyelec.com/NanoPiM5>) to download official image files (in the "01_Official images" directory). The table below lists all official images, the word 'XYZ' in image filename meaning:

- **sd**: Use it when you need to boot the entire OS from the SD card
- **eflasher**: Use it when you need to flash the OS to eMMC/UFS via TF card
- **usb**: Use it when you need to flash the OS to eMMC/UFS via USB

Icon	Image Filename	Version	Description	Kernel Version
	rk3576-XYZ-debian-trixie-gnome-wayland-desktop-6.1-arm64-YYYYMMDD.img.gz	trixie	Debian 13 with GNOME and Wayland with recommended software	6.1.y
	rk3576-XYZ-debian-trixie-gnome-wayland-minimal-6.1-arm64-YYYYMMDD.img.gz	trixie	Lightweight Debian 13 with GNOME and Wayland	6.1.y
	rk3576-XYZ-ubuntu-noble-lxqt-desktop-6.1-arm64-YYYYMMDD.img.gz	noble	Ubuntu 24.04 Desktop, LXQT desktop(X11), pre-installed mpv, supports hardware acceleration	6.1.y
	rk3576-XYZ-debian-trixie-core-6.1-arm64-YYYYMMDD.img.gz	trixie	Debian 13 Core , No desktop environment, command line only	6.1.y
	rk3576-XYZ-debian-bullseye-xfce-minimal-6.1-arm64-YYYYMMDD.img.gz	bullseye	Debian 11 Desktop, LXDE desktop, no pre-installed recommended software, supports hardware acceleration	6.1.y
	rk3576-XYZ-debian-bullseye-xfce-desktop-6.1-arm64-YYYYMMDD.img.gz	bullseye	Debian 11 Desktop, LXDE desktop, pre-installed mpv, smplayer and chromium browser, supports hardware acceleration	6.1.y
	rk3576-XYZ-ubuntu-noble-core-6.1-arm64-YYYYMMDD.img.gz	noble	64-bit Ubuntu image file based on Ubuntu core 24.04 64bit	6.1.y
	rk3576-XYZ-ubuntu-focal-lxqt-desktop-6.1-arm64-YYYYMMDD.img.gz	focal	Ubuntu 20.04 Desktop, LXQT desktop(X11), pre-installed mpv, smplayer and chromium browser, supports hardware acceleration	6.1.y
	rk3576-XYZ-friendlycore-focal-qt5-6.1-arm64-YYYYMMDD.img.gz	focal	FriendlyCore, command line only, pre-installed Qt5, based on Ubuntu core 20.04	6.1.y
	rk3576-XYZ-ubuntu-noble-gnome-desktop-6.1-arm64-YYYYMMDD.img.gz	noble	Ubuntu 24.04 with GNOME and Wayland with recommended software	6.1.y
	rk3576-XYZ-ubuntu-noble-gnome-minimal-6.1-arm64-YYYYMMDD.img.gz	noble	Lightweight Ubuntu 24.04 with GNOME and Wayland	6.1.y
	rk3576-XYZ-android14-YYYYMMDD.img.gz	14	Android 14 Tablet	6.1.y
	rk3576-XYZ-androidtv-YYYYMMDD.img.gz	14	Android 14 TV	6.1.y
	rk3576-XYZ-buildroot-6.1-arm64-YYYYMMDD.img.gz	--	Buildroot OS, base on Rockchip Linux SDK	6.1.y
	rk3576-XYZ-openmediavault-6.1-YYYYMMDD.img.gz	Synchrony	OpenMediaVault NAS system, base on Debian 13	6.1.y
	rk3576-XYZ-proxmox-6.1-YYYYMMDD.img.gz	8.2.7	Proxmox VE OS (Preview), base on Debian 12	6.1.y
	rk3576-XYZ-friendlywrt-25.12-YYYYMMDD.img.gz	25.12	FriendlyWrt, based on OpenWrt 25.12	6.1.y
	rk3576-XYZ-friendlywrt-25.12-docker-YYYYMMDD.img.gz	25.12	FriendlyWrt with Docker, based on OpenWrt 25.12	6.1.y
	nanopi_m5-sd-rocknix-YYYYMMDD.img.gz nanopi_r76s-sd-rocknix-YYYYMMDD.img.gz rk3576-eflasher-rocknix-6.1-YYYYMMDD.img.gz	--	ROCKNIX Linux Gaming OS (Preview), Supports only certain models. Supports booting from SD/eMMC/UFS, but does not support USB flashing.	6.1.y
Other Image				
	FriendlyWrt (Github Actions)	25.12,24.10	FriendlyWrt (https://github.com/friendlyarm/Actions-FriendlyWrt/releases)	6.1.y
	Alpine-Linux (Github Actions)	-	Alpine-Linux (https://github.com/friendlyarm/Actions-Alpine-Linux/releases)	6.1.y
	rk3576-eflasher-multiple-os-YYYYMMDD-50g.img.gz	-	It contains multiple OS image files, making it convenient for testing different operating systems, this image disables automatic flashing at startup; you will need to manually select the OS to flash, This image requires a high-capacity SD card (64 GB or larger);	

4.4.1.2 Tools (optional)

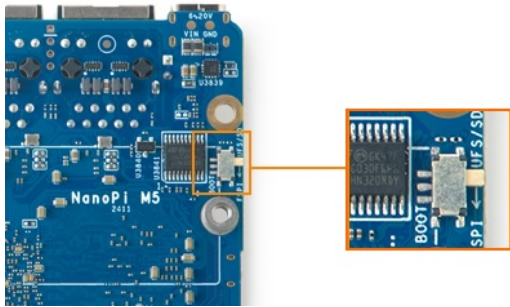
Visit download link (<http://download.friendlyelec.com/NanoPiM5>) to download tools (in the "05_Tools" directory).

Filename	Description
win32diskimager.rar	This program is designed to write a raw disk image to a removable device or backup a removable device to a raw image file
SD Card Formatter	A program (application) that allows easy and quick clear the SD card
RKDevTool_v3.37_for_window.zip	Rockchip flashing tool, for USB upgrade

4.4.2 Flashing the OS to the microSD card

Follow the steps below:

- Get an 8G microSD card;
- Visit download link (<http://download.friendlyelec.com/NanoPiM5>) to download image files (in the "01_Official images/01_SD card images" directory);
- Download the win32diskimager tool (in the "05_Tools" directory), or use your preferred tool;
- Extract the .gz format compressed file to get the .img format image file;
- Run the win32diskimager utility under Windows as administrator. On the utility's main window select your SD card's drive, the wanted image file and click on "write" to start flashing the SD card.
- Take out the SD and insert it to NanoPi-M5's microSD card slot;
- Set the BOOT switch on the back of the PCB to the 'UFS/SD' position:



- Power on NanoPi-M5 and it will be booted from your TF card, some models may require pressing the Power button to start;

4.4.3 Install OS to UFS

Note: Ensure that the BOOT switch on the back of the PCB remains set to the 'UFS/SD' position.

4.4.3.1 Option 1: Install OS via TF Card

This method firstly boots a mini Linux from a TF card and then automatically runs an EFlasher utility to install the OS to UFS. You can connect your system to an HDMI monitor and watch its progress.

This is optional. You can watch its progress by observing its LEDs as well:

Progress	SYS LED(Red)	LAN LED(Green)	WAN LED(Green)
Power On	Solid On	Off	Off
System Boot	Slow Flashing	Off	Off
Installation in Progress	Fast Flashing	Off	Off
Installation Done	Slow Flashing	Solid On	Solid On

By default, flashing starts automatically upon power-up, so be sure to back up the data in UFS. If you don't want it to start automatically, you can use image file with a filename containing the words 'multiple-os' and manually select the OS you want to flash on the interface.

4.4.3.1.1 Flash Official OS to UFS

Follow the steps below:

- Get an SDHC card with a minimum capacity of 8G
- Visit download link (<http://download.friendlyelec.com/NanoPiM5>) to download image files (in the "01_Official images/02_SD-to-eMMC images" directory) and win32diskimager tool (in the "05_Tools" directory);
- Extract the .gz format compressed file to get the .img format image file;
- Run the win32diskimager utility under Windows as administrator. On the utility's main window select your SD card's drive, the wanted image file and click on "write" to start flashing the SD card.
- Eject your SD card and insert it to NanoPi-M5's microSD card slot.
- Turn on NanoPi-M5, it will boot from the SD card and automatically run EFlasher to install the OS to the board's UFS.
- After flashing is complete, eject the SD card from NanoPi-M5, NanoPi-M5 will automatically reboot and boot from UFS.

4.4.3.1.2 Flash third party OS (Image file) to UFS

- Auto Install (Default Behavior)

- 1) Download an “eflasher” firmware from network drive (<http://download.friendlyelec.com/NanoPiM5/>)(in the "01_Official images/02_SD-to-eMMC images" directory), extract it and install it to a TF card ;
- 2) Eject and insert the TF card to your PC, after a “FriendlyARM” device shows up(Under Linux, it is a “FriendlyARM” directory), copy the image file ending with .raw or .gz into the directory (Note: if your file is in .img format, please rename it to .raw format).
- 3) Open the eflasher.conf file on the TF card, set “autoStart=” to the name of your image file, such as:

```
autoStart=openwrt-rockchip-armv8_nanopi-ext4-sysupgrade.img.gz
```

In addition to third-party image, official image files which with the '-sd-' word in the filename are also supported, for example: rk3NNN-sd-friendlywrt-24.10-YYYYMMDD.img.gz

- 4) Eject the TF card, insert the TF card to NanoPi-M5, power it on it will automatically install your firmware. You can watch the installation progress by observing the LEDs' status.

4.4.3.2 Option 2: Install OS on Web Page

Get a TF card which has been installed with FriendlyWrt, log in FriendlyWrt on the web page, click on “System” ->”UFS Tools”. Click on “Select file” to select your wanted image file, either an official image (filename containing '-sd-') or a third party image. The file should be a “.gz” or “.img” file.

After a file is selected, click on “Upload and Write” to start installing an OS.

600px

After installation is done, eject the SD card, the system will automatically reboot and load the OS from UFS. After the OS begins to load, if the system LED is flashing and the network LED is on, it means the the OS has loaded successfully. If the OS is FriendlyWrt, you can click on “Go to Homepage” to enter the homepage.

For official OS, you need select the file with the filename containing '-sd-', for example: rk3NNN-sd-friendlywrt-24.10-YYYYMMDD.img.gz, the compression file only supports the .gz format. If the file is too large, you can compress it into .gz format before uploading.

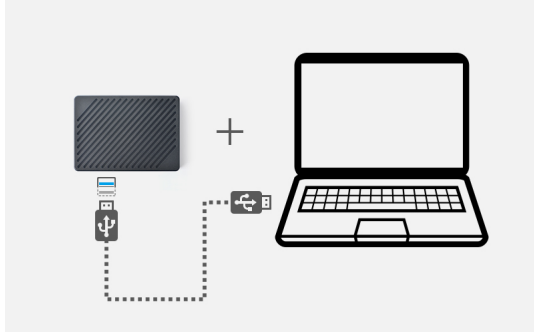
4.4.3.3 Option 3: Install OS via USB

4.4.3.3.1 Step 1: Download the required files and install the USB driver

Download the Rockchip USB driver (DriverAssitant_v5.12.zip) from the netdrive (<http://download.friendlyelec.com/NanoPiM5/>)tools directory, extract it, and install it; Download the required OS package from the netdrive (<http://download.friendlyelec.com/NanoPiM5/>) (located in the "01_Official images/03_USB upgrade images" directory), extract it, and you will find that the firmware directory contains the Rockchip development tool and the preconfigured configuration files;

4.4.3.3.2 Step 2: Connect NanoPi-M5 to PC and Enter Installation Mode

- 1) Press and hold the “Mask” key, power on the board. After the status LED has been on for at least 3 seconds, release the Mask key;
- 2) Use a USB A-to-A cable, connect NanoPi-M5 to a PC as follows;



4.4.3.3.3 Step 3: Launch RKDevTool

Double-click "RKDevTool.exe" in the image directory to start the Rockchip development tool. If the development board is properly connected to your PC, the RKDevTool interface will display "Found One MASKROM Device".

4.4.3.3.4 Step 4: Erase UFS

The erase operation is not mandatory—only required when the UFS contains a different system. If the device fails to boot after flashing, you should erase the UFS before reflashing.

Steps to erase the UFS:

- In the RKDevTool interface, open the "Advanced Function" tab.
- In the "Boot" field, select "MiniLoaderAll.bin", then click the "Download" button.
- Click “EraseAll” to wipe the UFS (Note: if the board includes SPI Flash, first check SATA and click “Switch Storage” before proceeding).

4.4.3.3.5 Step 5: Begin Flashing

Firmware usually comes in one of two formats: a single image file (common for third-party firmware), or multiple partition images (used by FriendlyELEC). Instructions are provided below for both formats:

- Option 1: Flash firmware composed of multiple partition images

In the RKDevTool interface, click "Download Image", then click the "Run" button. The flashing process will complete shortly, and the device will reboot automatically and boot the newly installed system from UFS;

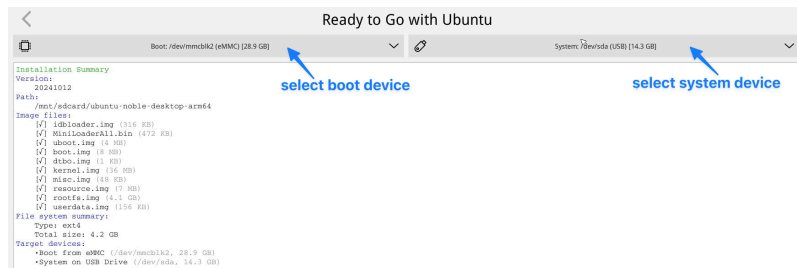
- Option 2: Flash firmware packaged as a single image file

In the RKDevTool interface, click "Upgrade Firmware", then click the "Firmware" button. Select the image file you wish to flash, click "Upgrade", and wait for the process to complete. Upon completion, the device will reboot and boot the newly installed system from UFS;

4.4.4 Installing the System to M.2 or USB Drive

You can use a TF card to boot the eFlasher system, allowing the boot and system to be installed on different storage devices. However, since the CPU doesn't support booting directly from M.2 and USB devices, the system can be installed on M.2 and USB devices, but the boot must still be installed on UFS or a TF card. Steps are as follows:

- Prepare a TF card with a capacity of 32GB or larger.
- Visit [the download link here](http://download.friendlyelec.com/NanoPiM5) to download the firmware file named XXXX-eflasher-multiple-os-YYYYMMDD-30g.img.gz (located in the "01_Official images/02_SD-to-eMMC images" directory).
- Flash the firmware to the TF card, connect the storage device you intend to use on NanoPi-M5, insert the TF card and power on, we need to perform the operations in the eFlasher GUI. If your NanoPi-M5 does not have a display interface, you can use VNC; refer to Using VNC to Operate eFlasher (https://wiki.friendlyelec.com/wiki/index.php/EFlasher#Remote_Control_eFlasher_via_VNC).
- In the eFlasher GUI, select the OS to install, and in the OS settings interface, choose the destination for boot installation (typically UFS), then choose the destination for system installation (options include UFS, M.2 hard drive, USB storage, etc.), as shown below:



- If no UFS is available, the TF card can serve as the boot by inserting another TF card into the USB port via a USB card reader and selecting it as the boot destination, enabling booting from the TF card with the system stored on the M.2 or USB drive.
- After flashing, eject the SD card from NanoPi-M5. If booting from UFS, NanoPi-M5 will automatically restart into the newly flashed system. If boot installation is on a TF card, power off, insert the boot TF card, and power on again.
- For a more detailed installation guide, please refer to this link (https://wiki.friendlyelec.com/wiki/index.php/EFlasher#Installing_the_System_to_M.2_or_USB_Drive_28Rockchip_Platform_Only.29).

5 Work with FriendlyWrt

5.1 Introduction to FriendlyWrt

FriendlyWrt is a customized system made by FriendlyElec based on an OpenWrt distribution. It is open source and well suitable for developing IoT applications, NAS applications etc.

FriendlyElec open sourced the automatic build project of FriendlyWrt on GitHub: Actions-FriendlyWrt (<https://github.com/friendlyarm/Actions-FriendlyWrt>). It builds the FriendlyWrt firmware with GitHub Actions and covers most of the NanoPi/NanoPC boards. In this project you can check which OpenWrt version, which kernel version and which packages the firmware is built from, and you can also fork it to build a customized firmware of your own. The built firmware is published on the releases (<https://github.com/friendlyarm/Actions-FriendlyWrt/releases>) page of the project, it contains two kinds of files: XYZ.img.gz is a firmware image which can be written to an SD card or the eMMC directly, images-XYZ.tgz is an upgrade package.

The FriendlyWrt firmware provided in the netdisk is kept in sync with the updates of this project.

5.2 First boot

For the first boot, the system needs to do the following initialization work:

- 1) Extended root file system
- 2) Initial setup (will execute /root/setup.sh)

So you need to wait for a while (about 2~3 minutes) to boot up for the first time, and then set FriendlyWrt, you can enter the ttyd terminal on the openwrt webpage, when the prompt is displayed as root@FriendlyWrt, it means the system has been initialized.

```
root@FriendlyWrt
```

5.3 Account & Password

The default password is password (empty password in some versions). Please set or change a safer password for web login and ssh login. It is recommended to complete this setting before connecting NanoPi-M5 to the Internet.

5.4 Login FriendlyWrt

Connect the PC to the LAN port of NanoPi-M5. If your PC without a built-in ethernet port, connect the LAN port of the wireless AP to the LAN port of NanoPi-M5, and then connect your PC to the wireless AP via WiFi, Enter the following URL on your PC's browser to access the admin page:

- <http://friendlywrt/>
- <http://192.168.2.1/>

- `http://[fd00:ab:cd::1]`

The above is the LAN port address of NanoPi-M5. The IP address of the WAN port will be dynamically obtained from your main router through DHCP.

5.5 Recommended security settings

The following settings are highly recommended to complete before connecting NanoPi-M5 to the Internet.

- Set a secure password
- Only allow access to ssh from lan, change the port
- Check the firewall settings

Set up as you wish.

5.6 Change LAN IP in LuCI

- 1) Click on Network → Interfaces, then click on the Edit button of the LAN Network;
- 2) In General Setup tab, input new IP address (for example: 192.168.11.1), click "Save" and then click "Save & Apply";
- 3) On the pop-up window with the title "Connectivity change", click "Apply and revert on connectivity loss";
- 4) Wait a moment, enter the new address in your computer's browser and login to FriendlyWrt;

5.7 Safe shutdown operation

Enter the "Services" -> "Terminal", enter the "poweroff" command and hit enter, wait until the led light is off, and then unplug the power supply.

5.8 Soft Factory Reset

Enter "System"->"Backup/Flash firmware", Click "Perform reset" Button, Your device's settings will be reset to defaults like when FriendlyWrt was first installed. You can also do this in the terminal:

```
firstboot && reboot
```

5.9 Install Software Packages

5.9.1 Package repositories

The repository list is `/etc/apk/repositories.d/distfeeds.list` (the old `/etc/opkg/distfeeds.conf` is gone), each line is the URL of a packages.adb index. You can edit it in "System" -> "Software" by clicking the configuration button, or in the ttyd terminal.

To switch to a mirror site in China, only the host name needs to be replaced, so the entries added by FriendlyElec (e.g. a local package directory) are kept:

```
sed -i -e 's#downloads.openwrt.org#mirrors.tuna.tsinghua.edu.cn/openwrt#g' /etc/apk/repositories.d/distfeeds.list
apk update
```

To switch back to the official server:

```
sed -i -e 's#mirrors.tuna.tsinghua.edu.cn/openwrt#downloads.openwrt.org#g' /etc/apk/repositories.d/distfeeds.list
apk update
```

- apk verifies the signature of the repository index, so make sure the system time is correct, otherwise apk update may fail;
- If https fails with a certificate error, install the ca-bundle and ca-certificates packages, or use http instead;

5.9.2 Update Package List

Before installing software packages update the package list:

```
apk update
```

5.9.3 List Available Packages

```
apk list
```

5.9.4 List Installed Packages

```
apk list --installed
```

5.9.5 Install Packages

```
apk add pkgname
```

5.9.6 Remove Packages

```
apk del pkgname
```

5.10 Configure the function of the user button

By default, the user button is configured to reboot the device, as shown below:

```
echo 'BTN_1 1 /sbin/reboot' >> /etc/triggerhappy/triggers.d/example.conf
```

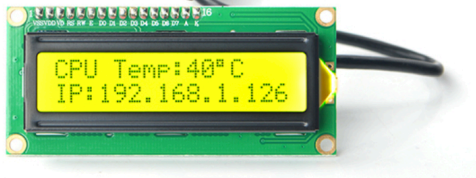
You can change its behavior by changing the configuration file above.

5.11 Some common issues of FriendlyWrt

- Unable to dial up
 - Go to "Network" -> "Firewall" and set "Inbound Data", "Outbound Data" and "Forwarding" in "WAN Zone" to "Accept";
- Dial-up successful, but no outgoing traffic
 - Go to "Services" -> "Terminal" and type "fw4 reload" to try to reload the firewall settings again;
- Unable to power on
 - Try to replace the power adapter and cable. It is recommended to use a power supply with specifications above 5V/2A;
 - Note that some fast chargers with Type-C interface will have a delay, it may take a few seconds to start providing power;
- When doing secondary routing, the computer cannot connect to the Internet
 - If your main network is IPv4, and NanoPi-M5 works in IPv6, the computer may not be able to connect to the Internet. It is recommended to switch the main route to IPv6;
- If you have questions or have better suggestions, please send an email to techsupport@friendlyarm.com;

5.12 Use USB2LCD to view IP and temperature

Plug the USB2LCD module to the USB interface of NanoPi-M5 and power on, the IP address and CPU temperature will be displayed on the LCD:



5.13 How to use USB WiFi

5.13.1 Check USB WiFi Device with Command Line Utility

(1) Click on "services>tytd" to start the command line utility

(2) Make sure no USB devices are connected to your board and run the following command to check if any USB devices are connected or not

```
lsusb
```

(3) Connect a USB WiFi device to the board and run the command again

```
lsusb
```

You will see a new device is detected. In our test the device's ID was 0BDA:C811

(4) Type your device's ID (in our case it was "0BDA:C811" or "VID_0BDA&PID_C811") in a search engine and you may find a device that matches the ID. In our case the device we got was Realtek 8811CU.

5.13.2 Configure a USB WiFi Device as AP

(1) Connect a USB WiFi device to the NanoPi-M5. We recommend you to use the following devices:

WiFi Chipset \ OS	Distro Support		AP Mode
	FriendlyWrt OpenWrt 19.07.5	Ubuntu Core Ubuntu 20.04 64-bit	
RTL8188CUS/8188EU 802.11n WLAN Adapter	Preinstalled driver	Yes	✗
RT2070 Wireless Adapter	Preinstalled driver	Yes	✗
RT2870/RT3070 Wireless Adapter	Preinstalled driver	Yes	✗
RTL8192CU Wireless Adapter	Preinstalled driver	Yes	✗
Ralink MT7601/MT7601U	Preinstalled driver	Yes	✗
5G USB WIFI RTL8821CU/RTL8811CU (VID_0BDA & PID_C811)	Plug and play, Access Point mode by default	Yes	✓
5G USB WIFI RTL8812BU (VID_0BDA & PID_B812)	Plug and play, Access Point mode by default	Yes	✓
5G USB WIFI RTL8812AU (VID_0BDA & PID_8812)	Plug and play, Access Point mode by default	Yes	✓
 5G USB WIFI MediaTek MT7662 (VID_0E8D & PID_7612)	Plug and play, Access Point mode by default	No	✓

Note: devices that match these VID&PIDs would most likely work.

(2) Click on "System>Reboot" and reboot your NanoPi-M5

(3) Click on "Network>Wireless" to enter the WiFi configuration page

(4) Click on "Edit" to edit the configuration

(5) On the "Interface Configuration" page you can set the WiFi mode and SSID, and then go to "Wireless Security" to change the password. By default the password is "password". After you make your changes click on "Save" to save

(6) After you change the settings you can use a smartphone or PC to search for WiFi

5.13.3 Common USB WiFi issues

1) It is recommended to plug in the usb wifi in the off state, then power it on, FriendlyWrt will automatically generate the configuration file /etc/config/wireless, if not, see if there is wlan0 by ifconfig -a, if there is no wlan0, usually there is no driver.

2) If ifconfig -a sees wlan0, but the hotspot is not working properly, try changing the channel and country code, an inappropriate country code can also cause the WiFi to not work.

3) Some USB WiFi (e.g. MTK MT7662) work in CD-ROM mode by default and need to be switched by usb_modeswitch, you can try to add usb_modeswitch configuration to the following directory: /etc/usb_modeswitch.d.

5.13.4 Change the default WiFi hotspot configuration

FriendlyWrt sets default parameters such as the country and the hotspot name for the WiFi modules it knows about, with the aim of being as plug-and-play as possible, but this does not guarantee that all modules will be compatible with this setting, you can change these behaviors by modifying the following file:

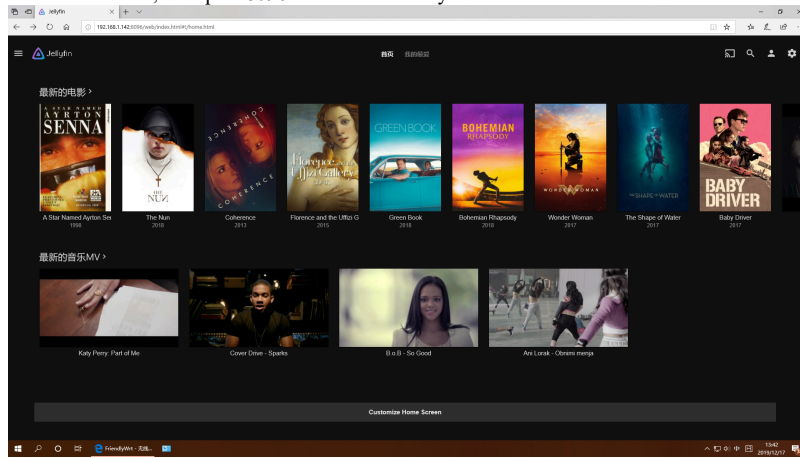
```
/lib/wifi/mac80211.uc
```

5.14 Work with Docker Applications

5.14.1 Work with Docker: Install JellyFin

```
mkdir -p /jellyfin/config
mkdir -p /jellyfin/videos
docker run --restart=always -d --name myjellyfin -p 8096:8096 -v /jellyfin/config:/config -v /jellyfin/videos:/videos jellyfin/jellyfin:latest
```

After installation, visit port 8096 and here is what you would find:



5.14.2 Work with Docker: Install Personal Nextcloud

```
mkdir /nextcloud -p
docker run -d -p 8888:80 --name nextcloud -v /nextcloud:/var/www/html/ --restart=always --privileged=true arm64v8/nextcloud
```

After installation, visit port 8888.

5.14.3 Docker FAQ and solutions

5.14.3.1 Unable to access the network services provided by the Docker container

Solution:

- Go to the "Firewall" settings and set "Forwarding" to "Accept";
- Turn off "Software Offload";

5.15 Mount smbfs

```
mount -t cifs //192.168.1.10/shared /movie -o username=xxx,password=yyy,file_mode=0644
```

5.16 Use sdk to compile the package

5.16.1 Install the compilation environment

Download and run the following script on 64-bit Ubuntu (version 22.04 or newer is recommended): How to setup the Compiling Environment on Ubuntu bionic (<https://github.com/friendlyarm/build-env-on-ubuntu-bionic>)

5.16.2 Download and decompress sdk from the network disk

The sdk is located in the toolchain directory of the network disk. The sdk of OpenWrt 25.12 is compressed with zstd (.tar.zst) and the toolchain is gcc 14.3.0, make sure the zstd package is installed on your PC (sudo apt install zstd):

```
tar xvf openwrt-sdk-*-rockchip-armv8_gcc-14.3.0_musl.Linux-x86_64.tar.zst
# If the path is too long, it will cause some package compilation errors, so change the directory name here
mv openwrt-sdk-*-rockchip-armv8_gcc-14.3.0_musl.Linux-x86_64 sdk
cd sdk
./scripts/feeds update -a
./scripts/feeds install -a
```

5.16.3 Compile the package

download the source code of the example (a total of 3 examples are example1, example2, example3), and copy to the package directory:

```
git clone https://github.com/mwarning/openwrt-examples.git
cp -rf openwrt-examples/example* package/
rm -rf openwrt-examples/
```

Then enter the configuration menu through the following command:

```
make menuconfig
```

In the menu, select the following packages we want to compile (actually selected by default):

```
"Utilities" => "example1"
"Utilities" => "example3"
```

```
"Network" => "VPN" => "example2"
```

execute the following commands to compile the three software packages:

```
make package/example1/compile V=99
make package/example2/compile V=99
make package/example3/compile V=99
```

After the compilation is successful, you can find the apk file (OpenWrt 25.12 no longer generates ipk files) in the bin directory, as shown below:

```
$ find ./bin -name "example*.apk"
./bin/packages/aarch64_generic/base/example3-1.0.0-r1.apk
./bin/packages/aarch64_generic/base/example1-1.0.0-r1.apk
./bin/packages/aarch64_generic/base/example2-1.0.0-r1.apk
```

5.16.4 Install the apk to NanoPi

You can use the scp command to upload the apk file to NanoPi:

```
cd ./bin/packages/aarch64_generic/base/
scp example*.apk root@192.168.2.1:/root/
```

Then use the apk command to install them, the "--allow-untrusted" option is required because the package files are not signed by the official repository:

```
cd /root/
apk add --allow-untrusted ./example3-1.0.0-r1.apk
apk add --allow-untrusted ./example1-1.0.0-r1.apk
apk add --allow-untrusted ./example2-1.0.0-r1.apk
```

5.17 Build FriendlyWrt using GitHub Actions

Please refre this link: <https://github.com/friendlyarm/Actions-FriendlyWrt>

6 Work with FriendlyCore

6.1 FriendlyCore User Account

- Non-root User:

```
User Name: pi
Password: pi
```

- Root:

```
User Name: root
Password: fa
```

6.2 Update Software Packages

```
$ sudo apt-get update
```

6.3 Setup Network Configurations

6.3.1 Set static IP address

By default "eth0" is assigned an IP address obtained via dhcp. If you want to change the setting you need to change the following file:

```
vi /etc/network/interfaces.d/eth0
```

For example if you want to assign a static IP to it you can run the following commands:

```
auto eth0
iface eth0 inet static
    address 192.168.1.231
    netmask 255.255.255.0
    gateway 192.168.1.1
```

6.3.2 Set a DNS

You also need to modify the following file to add the DNS configuration:

```
vi /etc/systemd/resolved.conf
```

For example, set to 192.168.1.1:

```
[Resolve]
DNS=192.168.1.1
```

Restart the systemd-resolved service with the following command:

```
sudo systemctl restart systemd-resolved.service
sudo systemctl enable systemd-resolved.service
```

6.3.3 Set up to use another network interface

To change the setting of "eth1" you can add a new file similar to eth0's configuration file under the /etc/network/interfaces.d/ directory.

6.4 Setup Wi-Fi

First, use the following command to check if Network-Manager is installed on your system:

```
which nmcli
```

If you have installed it, refer to this link to connect to WiFi: Use NetworkManager to configure network settings, If you do not have Network-Manager installed on your system, please refer to the following method to configure WiFi,

By default the WiFi device is "wlan0". You need to create a configuration file under "/etc/network/interfaces.d/" for WiFi:

```
vi /etc/network/interfaces.d/wlan0
```

Here is a sample wlan0 file:

```
auto lo
iface lo inet loopback
auto wlan0
iface wlan0 inet dhcp
wpa-driver wext
wpa-ssid YourWiFiSSID
wpa-ap-scan 1
wpa-proto RSN
wpa-pairwise CCMP
wpa-group CCMP
wpa-key-mgmt WPA-PSK
wpa-psk YourWiFiPassword
```

Please replace "YourWiFiSSID" and "YourWiFiPassword" with your WiFiESSID and password. After save and close the file you can connect to your WiFi source by running the following command:

```
sudo systemctl daemon-reload
sudo systemctl restart networking
```

After you power on your board it will automatically connect to your WiFi source.

Please note that if you use one TF card to boot multiple boards the WiFi device name will likely be named to "wlan1", "wlan2" and etc. You can reset it to "wlan0" by deleting the contents of the following file and reboot your board: /etc/udev/rules.d/70-persistent-net.rules

6.5 Install the kernel-header package

```
sudo dpkg -i /opt/archives/linux-headers-*.deb
```

6.6 Delete Qt5 and related files

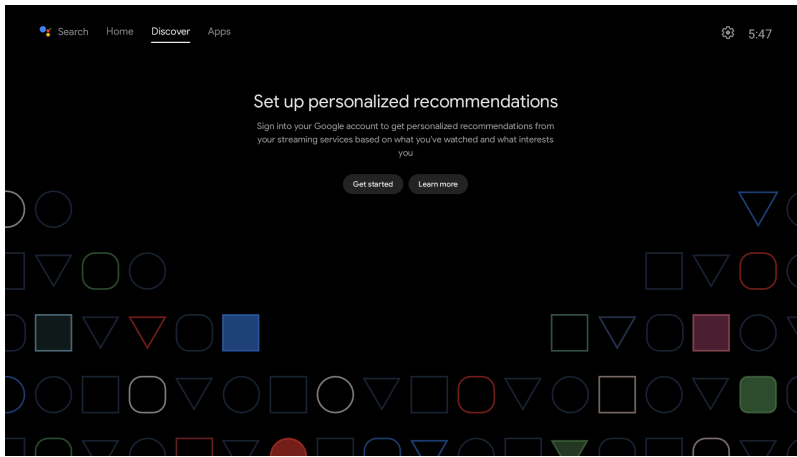
Execute the following commands:

```
su root
cd /
rm -rf usr/local/Trolltech/Qt-5.10.0-rk64one usr/local/Trolltech/Qt-5.10.0-rk64one-sdk usr/bin/setqt5env* usr/bin/qt5demo etc/qt5
rm -rf opt/{qt5-browser,Qt5_CinematicExperience,qt5-multi-screen-demo,qt5-nmapper,qt5-player,qt5-smarthome,QtE-Demo,qt5-qml-image-viewer,dual-camera}
```

7 Work with Android

Android include the following features:

- There are two versions, TV and Tablet;
- Support infrared remote control (only for models with Ir Receiver);
- Support Bluetooth remote control (requires USB or M.2 Bluetooth module);
- Support wired network;
- Support WiFi (requires external USB or M.2 WIFI module);
- Support video hard decoding;



7.1 WiFi models supported by Android

7.1.1 M.2 WiFi Module

- RTL8822CE

7.1.1.1 Usb Dongle

- RTL8821CU (Vid: 0BDA, Pid: C811) (Test sample:TP-Link TL-WDN5200H)
- RTL8812AU (Vid: 0BDA, Pid: 8812)
- MediaTek MT7662 (Vid: 0E8D, Pid: 7612) (Test sample:COMFAST CF-WU782AC V2)

7.2 Bluetooth models supported by Android

7.2.1 Bluetooth Adapters

- RTL8822CE
- RTL8761B
- CSR8510 A10 Bluetooth Dongle 0a12:0001

7.2.2 Bluetooth Remote

- Amazon Fire TV Remote

7.3 How to use ADB

7.3.1 USB connection

Please note: After turning on the ADB, the USB3 port will work in Device mode, if you need to connect a device such as a USB stick, you need to turn off the ADB and restart the board

In general, ADB is disabled by default, please follow the steps below to enable it:

- Connect your development board to your computer using a USB A-to-A data cable, referring to the figure below, be sure to connect it to the USB port closest to the edge:

7.3.2 For Android Tablet

- Go to Settings -> About tablet -> tap the "Build number" at the bottom of the screen 7 times in a row;
- Go to Settings -> System -> Advance -> Developer options, check USB-Debugging, and then reboot;
- To use ADB over the network, you need to connect to WiFi first, then enable Wireless debugging. In the prompt "Allow wireless debugging on the network", select "Always allow on this network", and then click "Allow".

7.3.3 For Android TV

- Click the Settings icon -> Device Preferences -> About -> tap the "Android TV OS build" at the bottom of the screen 7 times in a row;
- Click the Settings icon -> Device Preferences -> Developer options, check USB-Debugging, and then reboot;
- To use ADB over the network, on Android TV, which supports both WiFi and wired networks, enable network ADB by checking "Internet ADB" on the "Developer options" UI.

7.3.4 Using ADB

- Install ADB drivers and commands based on your operating system;
- Normally, the Android status bar will prompt "USB debugging connected", indicating that ADB has been enabled. Enter the following command on your computer to check the connection:

```
$ adb devices
List of devices attached
```

```
27f7a63caa3faf16      device
```

- Enter adb shell:

```
$ adb shell
nanopi3:/ $
```

7.3.5 Using ADB over the network

- On an Android Tablet, go to Settings -> System -> Advance -> Developer options, then click on Wireless debugging to view the IP address and port.
- The default network ADB port for Android TV is 5555.

Assuming the IP address and port displayed on the Wireless debugging interface are 192.168.1.167:45055, the ADB commands are as follows:

- To connect to the device:

```
$ adb connect 192.168.1.167:45055
```

connected to 192.168.1.167:45055

- To enter ADB shell:

```
$ adb shell
nanopi3:/ $
```

If there are multiple devices, use the -s parameter to specify the device's IP and port, as shown below:

```
$ adb -s 192.168.1.167:45055 shell
nanopi3:/ $
```

7.4 How to Change Default Launcher in Android TV

- Refer to the previous section to enable adb
- For example, install the third-party launcher Emotn UI via APK, visit the website <https://app.emotn.com/ui/> to download the APK, and then install it using ADB:

```
$ adb install com.oversea.aslauncher_1.0.9.0_5094.apk
Performing Streamed Install
Success
```

- After the installation is complete, launch it, and then enter the following ADB command to obtain its package name:

```
$ adb shell dumpsys window | grep mCurrentFocus
mCurrentFocus=Window{7a950fb u0 com.oversea.aslauncher/com.oversea.aslauncher.ui.main.MainActivity}
```

- As you can see, the package name of Emotn UI is com.oversea.aslauncher, set it as the default launcher:

```
$ adb shell pm set-home-activity com.oversea.aslauncher
Success
```

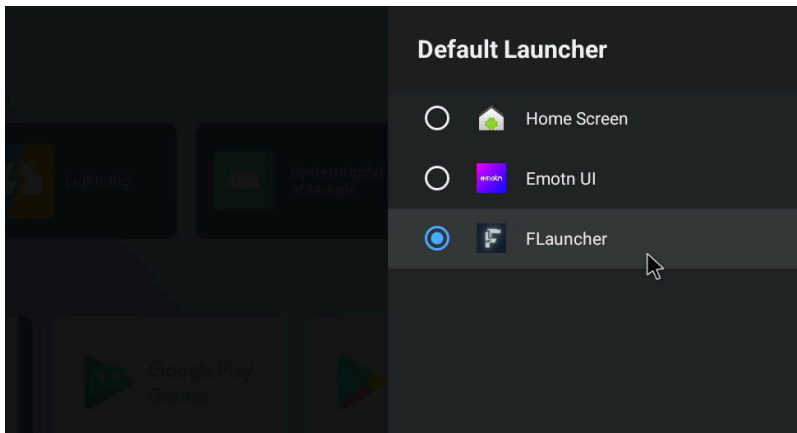
- Then comes the critical step, you need to disable the native launcher using the following command:

```
$ adb shell pm disable-user --user 0 com.google.android.tvlauncher
Package com.google.android.tvlauncher new state: disabled-user
```

- Finally, restart the device to see the effect, the device should boot directly into Emotn UI:

```
$ adb shell reboot
```

- From now on, if you want to install another launcher, you can switch between them through the GUI. for example, after installing FLAUNCHER, you can enter the following settings interface to set FLAUNCHER as the default launcher: Settings -> Device Preferences -> Advanced setting -> Default Launcher:



7.5 Wired networks on Android

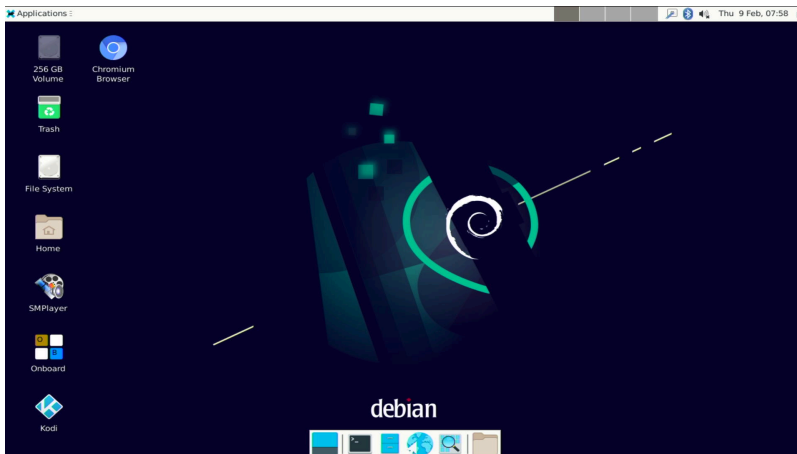
- Any network port can connect to the network via DHCP
- If you want to configure a static IP, only eth0 interface is supported
- Some applications may have compatibility issues and report no network connection error, but the network is actually connected

8 Work with Debian11 Desktop

8.1 Introduction to Debian11 Desktop

Debian11 Desktop is a light-weighted debian desktop system, it has the following features:

- Uses Xfce as default desktop;
- Mali GPU-based OpenGL support;
- Support Rockchip MPP video hard coding and hard decoding;
- Pre-installed mpv and smplayer, both support 4K video hardware decoding;
- Pre-installed Chromium browser, support vpu/gpu hardware acceleration (video hard decoding limited to h264/mp4 format);
- Compatible with Plex Server and Docker;



8.2 Account & Password

Regular Account:

User Name: pi
Password: pi

Root:

the root user account is disabled by default, you may configure the root password through the 'sudo passwd root' command.

8.3 View IP address

Since the Debian Bullseye hostname is the hardware model by default, you can use the ping command to get the IP address: `ping NanoPi-M5`

8.4 Connect to Debian via SSH

Run the following command `ssh pi@NanoPi-M5`

The default password is: pi

8.5 Update Software Packages

```
sudo apt-get update
```

8.6 Install x11vnc Server on Debian for Remote Access

8.6.1 Install x11vnc server

The following command to install x11vnc server:

```
sudo apt-get install x11vnc
```

8.6.2 Set your password

```
sudo x11vnc -storepasswd /etc/x11vnc.pwd
```

8.6.3 Setup x11vnc server with systemd auto start up

Create service configuration file:

```
sudo vi /lib/systemd/system/x11vnc.service
```

Let's copy and paste the following configuration into our newly create service file:

```
[Unit]
Description=Start x11vnc at startup.
Requires=display-manager.service
After=syslog.target network-online.target
Wants=syslog.target network-online.target

[Service]
Type=simple
ExecStart=/usr/bin/x11vnc -display :0 -forever -loop -noxdamage -repeat -rfbauth /etc/x11vnc.pwd -rfbport 5900 -shared -capslock -nomodtweak
ExecStop=/usr/bin/x11vnc -R stop
Restart=on-failure

[Install]
WantedBy=multi-user.target
```

The following commands to reload the systemd system and to enable and start the x11vnc service:

```
sudo systemctl daemon-reload
sudo systemctl enable x11vnc.service
sudo systemctl start x11vnc
```

8.6.4 Testing remote access

Start the VNC client software, input IP:5900 to connect:



8.7 Install the kernel-header package

```
sudo dpkg -i /opt/archives/linux-headers-*.deb
```

try to compile a kernel module:

```
sudo apt update
sudo apt install git gcc make bc
git clone https://github.com/carloscdias/hello-world-linux-module
cd hello-world-linux-module
```

```
make -j$(nproc)
make test
```

8.8 Change time zone

8.8.1 Check the current time zone

```
timedatectl
```

8.8.2 List all available time zones

```
timedatectl list-timezones
```

8.8.3 Set the time zone (e.g. Shanghai)

```
sudo timedatectl set-timezone Asia/Shanghai
```

8.9 Change startup LOGO and Wallpaper

8.9.1 Change startup LOGO

There are two ways to change the boot logo: rebuild the kernel on a Linux PC, or skip the PC entirely and let eFlasher write it while flashing.

8.9.1.1 Approach 1: On a Linux PC

Replace the following two files in the kernel source code directory and recompile the kernel :
kernel/logo.bmp
kernel/logo_kernel.bmp
Or use the script to operate, as shown below :

- Download scripts:

```
git clone https://github.com/friendlyarm/sd-fuse_rk3399.git -b kernel-4.19 --single-branch
cd sd-fuse_rk3399
```

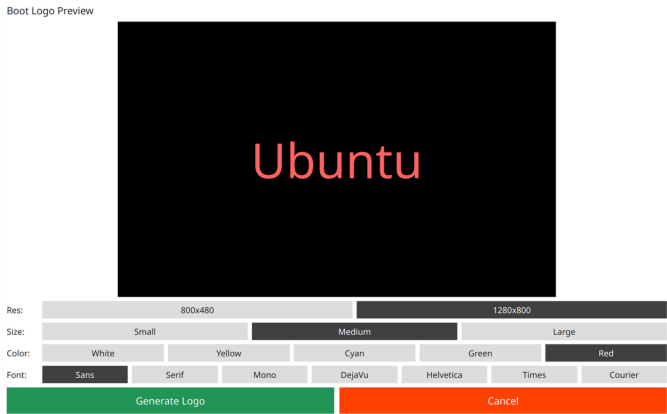
- Compile kernel and repackaging firmware

```
convert files/logo.jpg -type truecolor /tmp/logo.bmp
convert files/logo.jpg -type truecolor /tmp/logo_kernel.bmp
sudo LOGO=/tmp/logo.bmp KERNEL_LOGO=/tmp/logo_kernel.bmp ./build-kernel.sh debian-bullseye-desktop-arm64
sudo ./mk-sd-image.sh debian-bullseye-desktop-arm64
sudo ./mk-emmc-image.sh debian-bullseye-desktop-arm64
```

Note: If your system is not debian-bullseye-desktop-arm64, please specify according to the actual situation

8.9.1.2 Approach 2: No PC — with eFlasher

No Linux PC and no kernel rebuild needed: drop your logo.bmp (plus logo_kernel.bmp on Rockchip) into the matching OS directory on the eFlasher card, and it is written as the boot logo during flashing; you can also tap **Generate Boot Logo** in eFlasher to create one from text.



For the detailed steps, see the eFlasher page - Replace the Boot Logo.

8.9.2 Change Wallpaper

Modify the following configuration file:

```
/home/pi/.config/xfce4/xfconf/xfce-perchannel-xml/xfce4-desktop.xml
```

8.10 Soft Factory Reset

Execute the following command in a terminal:

```
sudo firstboot && sudo reboot
```

8.11 Start the program automatically at startup(For example Kodi)

Put the desktop file in the ~/.config/autostart/ directory, for example:

```
mkdir ~/.config/autostart/
cp /usr/share/applications/kodi.desktop ~/.config/autostart/
```

8.12 Disable auto-mounting

```
sudo systemctl mask udisks2
sudo reboot
```

8.13 Setup Chinese language and Input method

8.13.1 Setup Chinese language

Enter the following command and select 'zh_CN.UTF-8':

```
sudo dpkg-reconfigure locales
```

Add environment variables to .bashrc:

```
echo "export LC_ALL=zh_CN.UTF-8" >> ~/.bashrc
echo "export LANG=zh_CN.UTF-8" >> ~/.bashrc
echo "export LANGUAGE=zh_CN.UTF-8" >> ~/.bashrc
```

Reboot device:

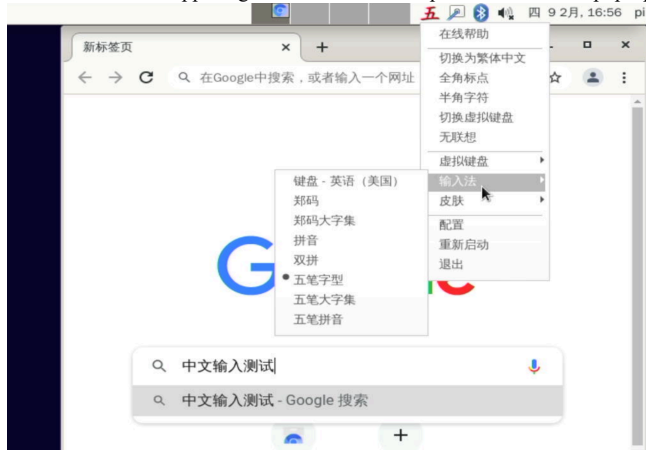
```
sudo reboot
```

8.13.2 Installing Chinese input method

Enter the following command to install fcitx and Pinyin input method:

```
sudo apt update
sudo apt-get install fcitx fcitx-pinyin
sudo apt-get install im-config
sudo apt-get install fcitx-table*
sudo apt-get install fcitx-ui-classic fcitx-ui-light
sudo apt-get install fcitx-frontend-gtk2 fcitx-frontend-gtk3 fcitx-frontend-qt4
sudo apt-get remove --purge scim* ibus*
sudo reboot
```

After reboot, press Ctrl+Space to switch between Chinese and English input methods, and the input method icon will appear in the upper right corner, right-click the input method icon in the upper right corner to switch input methods in the pop-up menu, as shown below:



8.14 Installing Plex Multimedia Server

Visit the Plex website: <https://www.plex.tv/media-server-downloads/>

On the download page, select the category "Plex Media Server", choose "Linux" for the platform and "Ubuntu(16.04+)/Debian(8+) - ARMv8" for the version, After downloading the deb package, use the dpkg command to install the package:

```
sudo dpkg -i plexmediaserver_1.31.0.6654-02189b09f_arm64.deb
```

After installation, login to the Plex server by typing the following URL into your computer browser: <http://IP地址:32400/web/>

8.15 Install Docker on Debian

Please refer to: [How to Install Docker on Debian](#)

8.16 How to test NPU

Please refer to: [NPU](#)

8.17 How to test VPU

Please refer to: [VPU](#)

8.18 WiFi Connection

8.18.1 Gui

Click on the icon on the top right in the Debian's main window, select your wanted WiFi hotspot and proceed with prompts

8.18.2 Console

Please visit: [Use NetworkManager to configure network settings](#)

8.19 Cancel auto-login

Edit file:

```
sudo vim /etc/lightdm/lightdm.conf
```

Comment out the following two lines (insert # in front of them):

```
autologin-user=pi
autologin-user-timeout=0
```

8.20 Test OpenGL ES

You can test it by clicking on the Terminator icon to start a commandline utility in the System Tools and run the following commands:

```
glmark2-es2
```

8.21 HDMI/DP LCD Resolution

Open the system's menu and go to Settings -> Display to customize your settings.

8.22 HiDPI and display scaling

Xfce supports HiDPI scaling which can be enabled using the settings manager: Go to Settings Manager > Appearance > Settings > Window Scaling and select 2 as the scaling factor.

Or Edit this file: `~/config/xfce4/xfconf/xfce-perchannel-xml/xsettings.xml`

8.23 Adjust HDMI overscan

Open the command line terminal and enter the command to operate, Note:

- 1) You need to login to the desktop;
- 2) If you are using ssh terminal, please use the same username as the desktop login. The default is pi. You cannot use the root user. you also need to assign the DISPLAY variable:

```
export DISPLAY=:0.0
```

8.23.1 Query which resolutions the display supports

```
xrandr -q
```

8.23.2 Set resolution

For example set to 1920X1080@60Hz:

```
xrandr --output HDMI-1 --mode 1920x1080 --refresh 60
```

8.23.3 Adjust the HDMI overscan

For example, the transformation scaling horizontal coordinates by 0.8, vertical coordinates by 1.04 and moving the screen by 35 pixels right and 19 pixels down:

```
xrandr --output HDMI-1 --transform 0.80,0,-35,0,1.04,-19,0,0,1
```

8.23.4 Automatic adjustment at boot

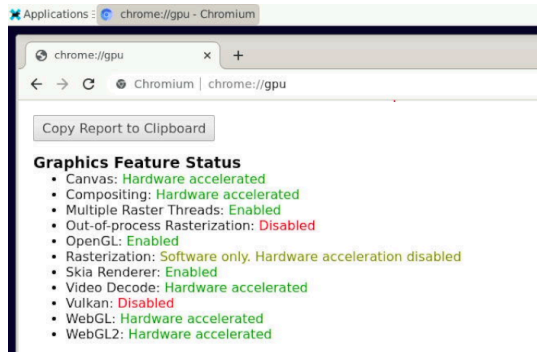
Edit ~/.config/autostart/lxrandr-autostart.desktop, Write the full xrandr command to the key at the beginning of "Exec=" as shown below :

```
[Desktop Entry]
Type=Application
Name=LXRandR autostart
Comment=Start xrandr with settings done in LXRandR
Exec=sh -c 'xrandr --output HDMI-1 --mode 1920x1080 --refresh 50 --transform 1.04,0,-35,0,1.05,-30,0,0,1'
```

8.24 Chromium web browser

8.24.1 GPU

Chromium web browser has enabled hardware acceleration by default, supports WebGL, and can view hardware acceleration details by entering the URL chrome://gpu, as shown below:



8.24.2 VPU

Play a video in the browser, then use fuser on the command line to view the mpp device node to confirm that the vpu interface is being called:

```
pi@FriendlyElec:~$ fuser /dev/mpp_service
/dev/mpp_service: 3258
```

If there is no content output from the fuser command, it means software decoding.

8.25 Test hardware encoding

```
mpi_enc_test -w 1920 -h 1080 -t 7 -f 0 -o test.h264 -n 300
export XDG_RUNTIME_DIR=/run/user/0
ffplay test.h264
```

8.25.1 Check Supported Hardware Decoding Formats

Enter about://gpu in your browser's address bar and scroll to the bottom of the page to view the "Video Acceleration Information" table. After playing a video, enter about://media-internals in your browser's address bar to check if hardware decoding was enabled for the most recent playback.

9 Work with Debian10 Desktop

- Refer to:
 - Debian Buster

10 Buildroot Linux

Buildroot is a simple, efficient and easy-to-use tool to generate embedded Linux systems through cross-compilation. It contains a boot-loader, kernel, rootfs, various libraries and utilities(e.g. qt, gstreamer, busybox etc).

FriendlyELEC's Buildroot is based on Rockchip's version which is made with linux-sdk and maintained with git. FriendlyELEC's version is synced with Rockchip's version;

- Rockchip's Buildroot: <https://github.com/rockchip-linux/buildroot>
- Buildroot's official site: <https://buildroot.org>

For a more detailed description of the Buildroot system, please refer to: Buildroot

11 How to Compile

11.1 Setup Development Environment

11.1.1 Method 1: Using docker to cross-compile

Please refre to docker-cross-compiler-novnc (<https://github.com/friendlyarm/docker-cross-compiler-novnc>)

11.1.2 Method 2: Setup build environment on the host machine

11.1.2.1 Install required packages

Install and run requirements ubuntu 20.04, install required packages using the following commands:

```
sudo apt-get -y update
sudo apt-get install -y sudo curl
sudo bash -c \
"$(curl -fsSL https://raw.githubusercontent.com/friendlyarm/build-env-on-ubuntu-bionic/master/install.sh)"
```

The following cross-compilers will be installed:

Version	Architecture	Compiler path	Purpose
4.9.3	armhf	/opt/FriendlyARM/toolchain/4.9.3	Can be used to build 32-bit ARM applications
6.4	aarch64	/opt/FriendlyARM/toolchain/6.4-aarch64	Can be used to build kernel 4.4
11.3	aarch64	/opt/FriendlyARM/toolchain/11.3-aarch64	Can be used to build kernel 4.19 or higher and U-Boot

11.1.2.2 Setting the compiler path

Based on the table in the previous section, select the appropriate version of the compiler and add the compiler's path to PATH. For example, if you want to use the 11.3 cross-compiler, edit ~/.bashrc using vi and add the following content to the end:

```
export PATH=/opt/FriendlyARM/toolchain/11.3-aarch64/bin:$PATH
export GCC_COLORS=auto
```

Run the ~/.bashrc script to make it effective in the current commandline. Note: there is a space after ".":

```
./~.bashrc
```

To verify if the installation was successful:

```
$ aarch64-linux-gcc -v
Using built-in specs.
COLLECT_GCC=aarch64-linux-gcc
COLLECT_LTO_WRAPPER=/opt/FriendlyARM/toolchain/11.3-aarch64/libexec/gcc/aarch64-cortexa53-linux-gnu/11.3.0/lto-wrapper
Target: aarch64-cortexa53-linux-gnu
Configured with: /home/cross/arm64/src/gcc/configure --build=x86_64-build_pc-linux-gnu --host=x86_64-build_pc-linux-gnu --target=aarch64-cortexa53-linux-gnu --prefix=/opt/FriendlyARM/toolch
Thread model: posix
Supported LTO compression algorithms: zlib
gcc version 11.3.0 (ctng-1.25.0-119g-FA)
```

11.2 Build Openwrt/Friendlywrt

11.2.1 Download FriendlyWrt 25.12 source code

```
mkdir friendlywrt25-rk3576
cd friendlywrt25-rk3576
git clone https://github.com/friendlyarm/repo --depth 1 tools
tools/repo init -u https://github.com/friendlyarm/friendlywrt_manifests -b master-v25.12 \
-m rk3576.xml --repo-url=https://github.com/friendlyarm/repo --no-clone-bundle
tools/repo sync -c --no-clone-bundle
```

11.2.2 First compilation step

```
./build.sh rk3576.mk # or rk3576-docker.mk
```

If a compilation error occurs, try single-threaded compilation with the following command:

```
(cd friendlywrt && make -j1 V=s)
```

For the docker version, try clearing the go cache before single-threaded compilation:

```
(cd friendlywrt && rm -rf tmp/go-build && make -j1 V=s)
```

All the components (including u-boot, kernel, and friendlywrt) are compiled and the sd card image will be generated, then execute the following command to generate the image file for installing the system into the emmc:

```
./build.sh emmc-img
```

After making changes to the project, the sd card image needs to be repackaged by running the following command:

```
./build.sh sd-img
```

11.2.3 Secondary compilation steps

```
cd friendlywrt
make menuconfig
rm -rf ./tmp
make -j$(nproc)
cd ../
./build.sh sd-img
./build.sh emmc-img
```

11.2.4 Build u-boot only

```
./build.sh uboot
```

11.2.5 Build kernel only

```
./build.sh kernel
```

11.2.6 Build friendlywrt only

```
./build.sh friendlywrt
```

Or go to the friendlywrt directory and follow the standard openwrt commands. If you get an error with the above command, try using the following command to compile in a single thread:

```
cd friendlywrt
make -j1 V=s
```

11.3 Build Buildroot

please refer to: Buildroot

11.4 Build Other Linux

11.4.1 Kernel and u-boot versions

Operating System	Kernel Version	U-boot version	Cross-compiler	Partition type	Packaging Tool	Kernel branch
buildroot	linux v6.1.y	u-boot v2017.09	11.3-aarch64	GPT (https://github.com/friendlyarm/sd-fuse_rk3576/blob/master/prebuilt/parameter.template)	sd-fuse (https://github.com/friendlyarm/sd-fuse_rk3576/tree/kernel-6.1.y)	nanopi6-v6.1.y (https://github.com/friendlyarm/rockchip/tree/nanopi6-v6.1.y)
openmediavault-arm64	linux v6.1.y			GPT (https://github.com/friendlyarm/sd-fuse_rk3576/blob/kernel-6.1.y/prebuilt/parameter-ext4.txt)	sd-fuse (https://github.com/friendlyarm/sd-fuse_rk3576/tree/kernel-6.1.y)	nanopi6-v6.1.y (https://github.com/friendlyarm/rockchip/tree/nanopi6-v6.1.y)
friendlycore-focal-arm64						
debian-trixie-core-arm64						
debian-bullseye-desktop-arm64						
debian-bullseye-minimal-arm64						
ubuntu-focal-desktop-arm64						
friendlywrt25						
friendlywrt25-docker						

- Kernel git repo: <https://github.com/friendlyarm/kernel-rockchip>
- U-boot git repo: <https://github.com/friendlyarm/uboot-rockchip>
- The cross-compile toolchain is located in the path: /opt/FriendlyARM/toolchain/
- The sd-fuse (https://github.com/friendlyarm/sd-fuse_rk3576/tree/kernel-6.1.y) is a helper script to make bootable SD card image.

11.4.2 Build kernel linux-v6.1.y

Clone the repository to your local drive then build:

```
git clone https://github.com/friendlyarm/kernel-rockchip --single-branch --depth 1 -b nanopi6-v6.1.y kernel-rockchip
cd kernel-rockchip
export PATH=/opt/FriendlyARM/toolchain/11.3-aarch64/bin/:$PATH
touch .scmversion
# Configuring the Kernel
# Option1: Load Linux configuration
make CROSS_COMPILE=aarch64-linux-gnu- ARCH=arm64 nanopi5_linux_defconfig
# Option2: Load Linux configuration with kvm support
make CROSS_COMPILE=aarch64-linux-gnu- ARCH=arm64 nanopi5_linux_defconfig kvm.config
# Option3, Load configuration for FriendlyWrt
make CROSS_COMPILE=aarch64-linux-gnu- ARCH=arm64 nanopi5_linux_defconfig friendlywrt.config
# Option4: Load panfrost configuration for Ubuntu 24.04 Desktop and enable KVM
make CROSS_COMPILE=aarch64-linux-gnu- ARCH=arm64 nanopi5_linux_defconfig kvm.config panfrost.config
# Optionally, if you want to change the default kernel config
make CROSS_COMPILE=aarch64-linux-gnu- ARCH=arm64 menuconfig
# Start building kernel
make CROSS_COMPILE=aarch64-linux-gnu- ARCH=arm64 nanopi5-images -j$(nproc)
# Start building kernel modules
mkdir -p out-modules && rm -rf out-modules/*
make CROSS_COMPILE=aarch64-linux-gnu- ARCH=arm64 INSTALL_MOD_PATH=$PWD/out-modules modules -j$(nproc)
make CROSS_COMPILE=aarch64-linux-gnu- ARCH=arm64 INSTALL_MOD_PATH=$PWD/out-modules modules_install INSTALL_MOD_STRIP=1
# Get kernel version
KERNEL_VER=$(make CROSS_COMPILE=aarch64-linux-gnu- ARCH=arm64 kernelrelease)
# Building r8125 module
export ETHTOOL_LEGACY_2500baseX=y
[ -d r8125 ] || git clone https://github.com/friendlyarm/r8125 -b main r8125
cd r8125
make CROSS_COMPILE=aarch64-linux-gnu- ARCH=arm64 KSRC=../ CONFIG_VENDOR_FRIENDLYARM=y CONFIG_WERROR=n -j$(nproc)
aarch64-linux-gnu-strip --strip-unneeded r8125.ko
mkdir -p ../out-modules/lib/modules/${KERNEL_VER}/extra/
cp r8125.ko ../out-modules/lib/modules/${KERNEL_VER}/extra/ -afv
cd ..
unset ETHTOOL_LEGACY_2500baseX
# Generate modules.dep
[ ! -f "$PWD/out-modules/lib/modules/${KERNEL_VER}/modules.dep" ] && depmod -b $PWD/out-modules -E Module.symvers -F System.map -w ${KERNEL_VER}
```

The generated files:

kernel.img	resource.img	boot.img This img is deprecated	The kernel modules are located in the out-modules directory
------------	--------------	---------------------------------	---

Run your build:

Please refer to #Running the build

11.4.3 Build u-boot v2017.09

Clone the repository to your local drive then build:

```
git clone https://github.com/friendlyarm/rkbin --single-branch --depth 1 -b nanopi_m5
git clone https://github.com/friendlyarm/uboot-rockchip --single-branch --depth 1 -b nanopi5-v2017.09
export PATH=/opt/FriendlyARM/toolchain/11.3-aarch64/bin/:$PATH
cd uboot-rockchip/
./make.sh nanopi_m5
```

After the compilation, the following files will be generated:

uboot.img	rk3576_spl_loader_xx.yy.zzz.bin (aka MiniLoaderAll.bin)
-----------	---

Run your build:

Please refer to #Running the build

11.4.4 Running the build

11.4.4.1 Install to target board

rk3576 uses GPT partitions by default, you can use the dd command, but be careful to choose the right output device:

- The SD/TF Card device node: /dev/mmcblk0
- The eMMC device node: /dev/mmcblk2

Use the 'parted' command to view the partition layout:

```
parted /dev/mmcblk2 print
```

Sample outputs:

```
Model: MMC A3A551 (sd/mmc)
Disk /dev/mmcblk2: 31.0GB
Sector size (logical/physical): 512B/512B
```

```
Partition Table: gpt
Disk Flags:

Number  Start   End     Size    File system  Name      Flags
 1      8389kB  12.6MB  4194kB                uboot
 2      12.6MB  16.8MB  4194kB                misc
 3      16.8MB  21.0MB  4194kB                dtbo
 4      21.0MB  37.7MB  16.8MB                resource
 5      37.7MB  79.7MB  41.9MB                kernel
 6      79.7MB  113MB   33.6MB                boot
 7      113MB   147MB   33.6MB                recovery
 8      147MB   31.0GB  30.9GB  ext4              rootfs
```

as shown above, the resource partition is located at 4 and the kernel partition is located at 5. Use the dd command to write the resource.img and kernel.img files to these partitions, the commands are as follows:

```
dd if=resource.img of=/dev/mmcblk2p4 bs=1M
dd if=kernel.img of=/dev/mmcblk2p5 bs=1M
```

If you want to update u-boot:

```
dd if=u-boot.img of=/dev/mmcblk2p1 bs=1M
```

To update new driver modules, copy the newly compiled driver modules to the appropriate directory under /lib/modules.

11.4.4.2 Packaging and creating an SD image

To create a new OS image file, you need to use the "sd-fuse" packaging tool.

"sd-fuse" is a collection of scripts that can be used to create bootable SD card images for FriendlyElec boards. Its main features include:

- Creation of root filesystem images from a directory
- Building of bootable SD card images
- Simple compilation of kernel, U-Boot, and third-party drivers

Please click on the following link to find out more:

Kernel version	Packaging Tool
linux v6.1.y	sd-fuse_rk3576 (https://github.com/friendlyarm/sd-fuse_rk3576/tree/kernel-6.1.y)

11.4.4.3 USB flashing

11.4.4.3.1 Linux

Reboot the board and enter loader mode with the following command:

```
sudo reboot loader
```

To flash U-Boot and kernel using the "upgrade_tool_v2.17_for_linux" tool, please use the following command:

```
sudo upgrade_tool di -k kernel.img
sudo upgrade_tool di -re resource.img
sudo upgrade_tool di -u u-boot.img
sudo upgrade_tool RD
```

Note: "upgrade_tool" is a command-line tool provided by Rockchip for Linux operating systems (Linux_Upgrade_Tool).

11.5 Build the code using scripts

11.5.1 Download scripts and image files

```
git clone https://github.com/friendlyarm/sd-fuse_rk3576.git --single-branch -b kernel-6.1.y
cd sd-fuse_rk3576
tar xvfz /path/to/netdrive/03_Partition\ image\ files/friendlycore-focal-arm64-images.tgz
```

11.5.2 Compile the kernel

Download the kernel source code and compile it. the relevant image files in the friendlycore-focal-arm64 directory will be automatically updated, including the kernel modules in the file system:

```
git clone https://github.com/friendlyarm/kernel-rockchip --depth 1 -b nanopi6-v6.1.y kernel-rk3576
KERNEL_SRC=$PWD/kernel-rk3576 ./build-kernel.sh friendlycore-focal-arm64
```

11.5.3 Compile the kernel headers

```
git clone https://github.com/friendlyarm/kernel-rockchip --depth 1 -b nanopi6-v6.1.y kernel-rk3576
MK_HEADERS_DEB=1 BUILD_THIRD_PARTY_DRIVER=0 KERNEL_SRC=$PWD/kernel-rk3576 ./build-kernel.sh friendlycore-focal-arm64
```

11.5.4 Compile the uboot

Download the uboot source code and compile it. the relevant image files in the friendlycore-focal-arm64 directory will be automatically updated:

```
git clone https://github.com/friendlyarm/uboot-rockchip --depth 1 -b nanopi5-v2017.09
UBOOT_SRC=$PWD/uboot-rockchip ./build-uboot.sh friendlycore-focal-arm64
```

11.5.5 Generate new image

Repackage the image file in the friendlycore-focal-arm64 directory into sd card image:

```
./mk-sd-image.sh friendlycore-focal-arm64
```

After the command is completed, the image is in the out directory, you can use the dd command to make the SD boot card, for example:

```
dd if=out/rk3576-sd-friendlycore-focal-5.10-arm64-YYYYMMDD.img of=/dev/sdX bs=1M
```

11.6 Building AOSP from source

11.6.1 Hardware and Software Requirements

- Your computer should have at least 32GB of RAM and 300GB of disk space. We recommend using a machine with 64GB of RAM and a large-capacity, high-speed SSD, and we do not recommend using virtual machines.
- If you encounter compilation errors, they may be caused by problems with the compilation environment. We recommend using the following Docker container for compilation: docker-cross-compiler-novnc (<https://github.com/friendlyarm/docker-cross-compiler-novnc>).

11.6.2 Download source from the netdrive

Netdisk URL: Click here (<http://download.friendlyelec.com/NanoPiM5>)

File location on netdisk: "07_Source codes/rk35xx-android14-xxxxxxx-YYYYMMDD.tgz" (YYYYMMDD represents the date of the package, and xxxxxxx represents the final commit-id)

Unzip and fetch updates:

```
tar xzf '/path/to/netdisk/07_Source codes/rk35xx-android14-xxxxxxx-YYYYMMDD.tgz'
cd rk35xx-android14
[ -d .git ] && git pull
```

11.6.3 Tablet profile build (First Build)

```
echo "ROCKCHIP_DEVICE_DIR := device/rockchip/rk3576/nanopi5" > .rockchip_device.mk
## If you need to build WiFi Display, please uncomment and run the following command:
# echo "ROCKCHIP_WIFI_DISPLAY := yes" >> .rockchip_device.mk
## If you need to build a version with pre-installed Google apps, please uncomment and run the following command:
# echo "INSTALL_GAPPS_FOR_TESTING := yes" >> .rockchip_device.mk
## If you need to add support for the Quectel EM05/EC20 4G module, please uncomment and run the following command:
# echo "BUILD_WITH_EC20 := yes" >> .rockchip_device.mk
## Initialize the environment and start building:
export BUILD_NUMBER=$(date +"%Y%m%d")1
. setenv.sh
./build.sh -FMu
```

If you want to generate an OTA update package, please add the `-O` option when running the build.sh command:

```
./build.sh -FMuO
```

If a build error occurs, try the following commands:

```
# Continue building with a single thread
make -j1
# If no errors occur, run build.sh again to package the images
./build.sh -FMu
```

11.6.4 TV profile build (First Build)

```
echo "ROCKCHIP_DEVICE_DIR := device/rockchip/rk3576/nanopi5_box" > .rockchip_device.mk
## If you need to build a version with pre-installed Google apps, please uncomment and run the following command:
# echo "INSTALL_GAPPS_FOR_TESTING := yes" >> .rockchip_device.mk
## Initialize the environment and start building:
export BUILD_NUMBER=$(date +"%Y%m%d")1
. setenv.sh
./build.sh -FMu
```

If a build error occurs, try the following commands:

```
# Continue building with a single thread
make -j1
# If no errors occur, run build.sh again to package the images
./build.sh -FMu
```

11.6.5 Second build

```
. setenv.sh
./build.sh -FMu
```

If you need to clean before building:

```
. setenv.sh
make installclean
rm -rf out/dist out/target/product/nanopi3/*
./build.sh -FMu
```

11.6.6 Running your AOSP build

After the Android compilation is completed, the image file will be stored in the rockdev/Image-aosp_nanopi3 subdirectory of the Android source code directory.

11.6.6.1 USB Flashing

Use the rockchip tool to flash the following file: rockdev/Image-aosp_nanopi3/update.img

11.6.6.2 SD-to-eMMC Flashing

Refer to the following steps:

- 1) Insert the SD card of the eflasher system into the host;
- 2) Copy the files in the rockdev/Image-aosp_nanopi3 directory to the android14 or android14 directory in the FRIENDLYARM partition of the SD card:

```
sudo cp -af parameter.txt config.cfg MiniLoaderAll.bin uboot.img \
dtbo.img vbmeta.img boot.img \
misc.img pcba_small_misc.img pcba_whole_misc.img \
baseparameter.img super.img /media/$USER/FriendlyARM/android14
```

- 3) The Image-aosp_nanopi3 directory does not contain ****idbloader.img****. You can download it from here:

```
cd /media/$USER/FriendlyARM/android14
[ -f idbloader.img ]
```

- 4) Insert the SD card into NanoPi M5 and re-flash;

11.6.7 Pack the new SD Image

```
git clone https://github.com/friendlyarm/sd-fuse_rk3576.git
SDFUSE=$PWD/sd-fuse_rk3576
mkdir $SDFUSE/android14

cd /path/to/rk35xx-android14/rockdev/Image-aosp_nanopi3
cp -af parameter.txt config.cfg MiniLoaderAll.bin uboot.img \
dtbo.img vbmeta.img boot.img \
misc.img pcba_small_misc.img pcba_whole_misc.img \
baseparameter.img super.img $SDFUSE/android14

wget http://112.124.9.243/dvdfilerk3576/tools/idbloader.img -O $SDFUSE/android14/idbloader.img

cd $SDFUSE/
./mk-sd-image.sh android14

# Extract the eFlasher system image from the downloaded archive
tar xvf /path/to/netdrive/03_Partition\ image\ files/emmc-flasher-images.tgz
./mk-emmc-image.sh android14
```

For more information, please refer to #Packaging and creating an SD image

12 Using On-Board Hardware Resources

12.1 VPU

Please refer to NPU

12.2 NPU

Please refer to NPU

12.3 DTS files

Please refer to DTS files

12.4 GPIO

Please refer to GPIO Programming: Using the libgpiod Library

12.5 Serial Port

The kernel has enabled the following serial ports for testing by default (should be based on kernel dts configuration), to use more serial ports you can configure the kernel dts yourself:

CPU	Model	Port	Device Node
RK3576	NanoPi-R76S	UART6	/dev/ttyS6
RK3576	NanoPi-M5	UART8	/dev/ttyS8

12.6 RTC

You can access RTC by calling APIs under the "/sys/class/rtc/rtc0/" directory. For instance you can check the current RTC time by running the following commands:

```
cat /sys/class/rtc/rtc0/date
# 2018-10-20
cat /sys/class/rtc/rtc0/time
# 08:20:14
```

12.7 Watch dog

Operating the watchdog is simple: open the device /dev/watchdog and write a character to it periodically. If the system fails to write, the device will automatically reboot after a timeout:

```
int fd = open("/dev/watchdog", O_WRONLY);
write(fd, "a", 1);
```

13 Usage of RKLLM and Large Language Model Testing

13.1 Demo: Model Inference in a Docker Container

Using the FriendlyWrt system as an example, we run the RKLLM application in a Docker environment to demonstrate how to deploy a large language model onto a development board.

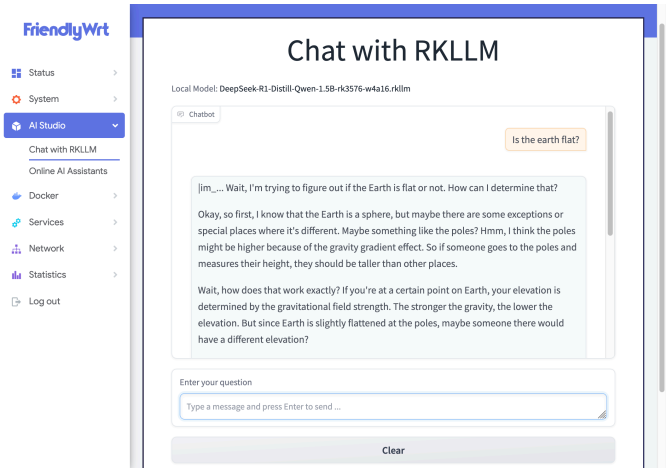
Visit download link (<http://download.friendlyelec.com/NanoPiM5>) to download official image files (in the "09_Other files/friendlywrt-ai" directory):

Image Files	
rk3576-sd-friendlywrt-24.10-ai-YYYYMMDD.img.gz	OS image booting from SD card
rk3576-eflasher-friendlywrt-24.10-ai-YYYYMMDD.img.gz	Image used to flash to eMMC

Note: Due to the system having built-in Docker applications and large language models, the first boot will take a longer time (about 160 seconds) for initialization. Once completed, the following files can be seen:

```
root@FriendlyWrt:~# ls /opt/rkllm_chat
docker-compose.yml  model
```

The UI interface is shown below:



13.2 Switching Large Language Models

In the pre-configured image mentioned above, the default large language model is DeepSeek-R1-Distill-Qwen-1.5B-rk3576-w4a16.rkllm. You can switch to another model using the following method:

- Copy the RKLLM model file (in RKLLM format) to the /opt/rkllm_chat/model directory.
- Modify the --rkllm_model_path parameter in the /opt/rkllm_chat/docker-compose.yml file to point to the RKLLM model file.
- Restart the Docker container with the commands:

```
# cd /opt/rkllm_chat
# docker-compose down
# docker-compose up -d
```

We provide several pre-converted RKLLM model files, which can be found in the following path on the network drive: "09_Other files/RKLLM models". The network drive link is: <http://dl.friendlyelec.com/rk3576>.

13.3 Currently Supported Models

- InternLM2-1.8B (https://huggingface.co/internlm/internlm2-1_8b)
- TinyLLAMA-1.1B (<https://huggingface.co/TinyLlama/TinyLlama-1.1B-Chat-v1.0>)
- Qwen2.5-1.5B (<https://huggingface.co/Qwen/Qwen2.5-1.5B>)
- Qwen2.5-3B (<https://huggingface.co/Qwen/Qwen2.5-3B>)
- DeepSeek-R1-Distill-Qwen-1.5B (<https://huggingface.co/deepseek-ai/DeepSeek-R1-Distill-Qwen-1.5B>)
- DeepSeek-R1-Distill-Qwen-7B (<https://huggingface.co/deepseek-ai/DeepSeek-R1-Distill-Qwen-7B>)
- MiniCPM3-4B (<https://huggingface.co/openbmb/MiniCPM3-4B>)
- Llama-3.2-1B-Instruct (<https://www.modelscope.cn/models/LLM-Research/Llama-3.2-1B-Instruct>)
- Gemma-2B (<https://www.huggingface.co/google/gemma-2b>)
- Gemma-2-9B-IT (<https://www.huggingface.co/google/gemma-2-9b-it>)
- ChatGLM3-6B (<https://huggingface.co/THUDM/chatglm3-6b>)

For more information on supported models, please refer to the official list from Rockchip: <https://github.com/airockchip/rknn-llm>

13.4 How to Convert a Model

For detailed information on using the NPU and RKLLM, as well as model conversion, please refer to NPU/zh.

14 Make a master board: clone your system and apps onto other boards

Once you have a board set up the way you want (apps installed, configs tweaked), you often need to replicate it exactly onto other boards for batch deployment. There are two approaches — pick whichever fits your setup:

- **Approach 1: build it on a Linux PC** — back up the board's root filesystem, then repack it into a flashable image on a Linux PC with the sd-fuse scripts.
- **Approach 2: no PC — build it directly with eFlasher** — no Linux PC needed; use eFlasher right on the board to extract the running system from the on-board storage into a flashable image.

14.1 Approach 1: build it on a Linux PC (sd-fuse)

14.1.1 Backup rootfs

Run the following commands on your target board. These commands will back up the entire root partition:

```
sudo passwd root
su root
cd /
tar --warning=no-file-changed -cvpzf /rootfs.tar.gz \
--exclude=/rootfs.tar.gz --exclude=/var/lib/docker/runtimes \
--exclude=/etc/firstuser --exclude=/etc/friendlyelec-release \
--exclude=/usr/local/first_boot_flag --one-file-system /
```

Note: if there is a mounted directory on the system, an error message will appear at the end, which can be ignored.

14.1.2 Making a bootable image from a root filesystem

Run the following script on your Linux PC host, we'll only mention "debian-trixie-core-arm64 os" for brevity, but you can apply the same process for every linux OS.

```
su root
git clone https://github.com/friendlyarm/sd-fuse_rk3576 --single-branch -b kernel-6.1.y
cd sd-fuse_rk3576
tar xvfz /path/to/netdrive/03_Partition\ image\ files/debian-trixie-core-arm64-images.tgz
tar xvfz /path/to/netdrive/03_Partition\ image\ files/emmc-eflasher-images.tgz
scp pi@BOARDIP:/rootfs.tar.gz /rootfs.tar.gz
mkdir rootfs
tar xvfz /rootfs.tar.gz -C rootfs --numeric-owner --same-owner
./build-rootfs-img.sh rootfs debian-trixie-core-arm64
./mk-sd-image.sh debian-trixie-core-arm64
./mk-emmc-image.sh debian-trixie-core-arm64 autostart=yes
```

14.2 Approach 2: no PC — build it directly with eFlasher

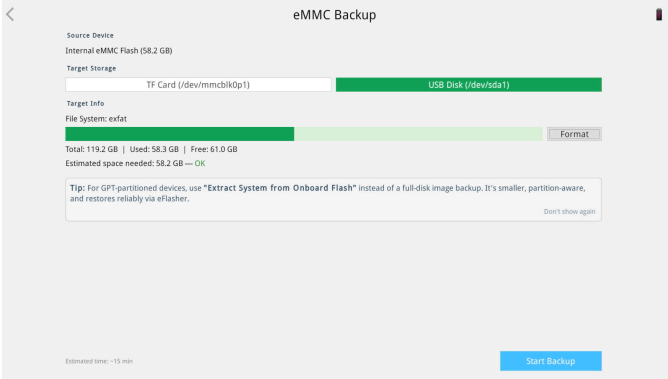
Recent eFlasher versions can extract the system right on the board: boot the board from a TF card flashed with the xxxx-eflasher-multiple-os-YYYYMMDD.img.gz firmware, and you can turn the system running on the on-board storage (eMMC / UFS) into a re-flashable image without any Linux PC. eFlasher offers two ways — pick the one that matches your use case:

14.2.1 Bit-by-bit copy (whole-drive, Ghost-style backup)

Sector-level raw read/write of the eMMC (or UFS), cloning the entire drive verbatim into a single backup file — similar to Ghost on Windows.

- **Pros:** works for any OS (including Android), filesystem-agnostic; even raw data such as unpartitioned sectors and vendor storage is captured verbatim.
- **Cons:** an uncompressed backup is as large as the total storage capacity, and can only be restored to a target no smaller than the source.

On the eFlasher main screen, tap **Advanced eMMC Backup** to enter the bit-by-bit backup view:



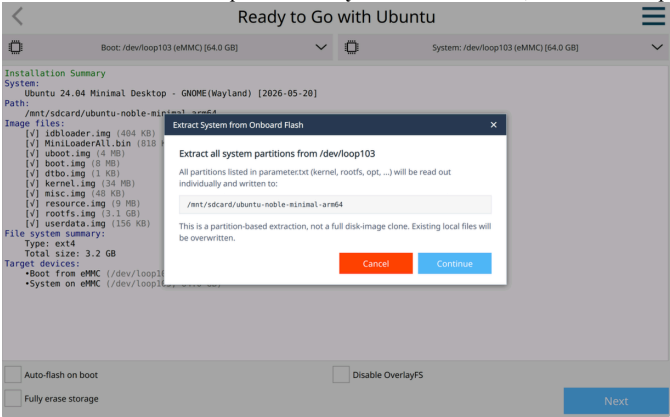
14.2.2 File-based reverse extraction (recommended for master images)

Reconstructs a flashable image package from the live filesystem while preserving the original partition layout.

- **Pros:** backup and restore speeds are close to flashing a stock firmware; adapts across eMMCs of different sizes (e.g. a large eMMC in development, a smaller one in production); with an OverlayFS layout every user-side change is lifted and preserved, making it ideal for building a master image.
- **Cons:** only Rockchip-platform Linux systems using a GPT partition table are supported.

On the eFlasher main screen, tap the OS icon that matches the system already installed on the eMMC to open the **Ready to Go with ...** view, then use the top-right menu and

pick **Extract System from Onboard Flash**, following the on-screen prompts:



For the full step-by-step of both eFlasher methods (restore, auto-restore on boot, hook scripts, etc.), see the EFlasher page.

15 Common Linux-based operating system operations

15.1 Using ADB on Linux Systems

15.1.1 Enabling ADB in Buildroot System

Enable on Startup

```
mv /etc/init.d/K50usbdevice.sh /etc/init.d/S50usbdevice.sh
reboot
```

Enable Temporarily

```
usbdevice-wrapper start
```

15.1.2 Enabling ADB in Ubuntu and Debian Systems

Enable on Startup

```
sudo systemctl enable usbdevice
sudo reboot
```

Enable Temporarily

```
usbdevice-wrapper start
```

15.1.3 How to Connect

When using ADB, the port connected to the computer is the same as the USB flashing port.

15.2 Install Kernel Headers

To install the .deb file located in the /opt/archives directory:

```
sudo dpkg -i /opt/archives/linux-headers-*.deb
```

To download and update the kernel header files online:

```
wget http://112.124.9.243/archives/rk3576/linux-headers-$(uname -r)-latest.deb
sudo dpkg -i ./linux-headers-latest.deb
```

You can visit <http://112.124.9.243/archives/rk3576> to see which kernel deb packages are available.

15.3 Setting Kernel Boot Parameters

15.3.1 When the system needs to be installed on eMMC/UFS

Flash the firmware file XXXX-eflasher-multiple-os-YYYYMMDD-30g.img.gz to a TF card, then insert the TF card into your computer. Windows will usually recognize the TF card partition automatically (formatted as exFAT). For Linux or Mac users, manually mount the first partition of the TF card. Assuming the TF card's device name is /dev/sdX, mount /dev/sdX1.

Edit the info.conf configuration file in the OS directory on the TF card, adding the bootargs-ext parameter. For example:

```
bootargs-ext=rockchipdrm.fb_max_sz=2048
```

To remove a specified parameter, set it to empty. For example, to remove the userdata parameter:

```
bootargs-ext=userdata=
```

After editing, use this TF card to flash the system to eMMC/UFS Only.

To set kernel boot parameters during the creation of a mass production card, refer to the following script (example for RK3588): https://github.com/friendlyarm/sd-fuse_rk3588/blob/kernel-6.1.y/test/test-custom-bootargs.sh

15.3.2 Change kernel boot parameters by compiling the kernel

Search for the bootargs field in the kernel source directory and modify the DTS file of the target development board (click DTS files to view the corresponding DTS file for the board):

```
grep -R "bootargs" arch/arm64/boot/dts/rockchip
```

Then recompile the kernel and update resource.img.

15.4 View the current kernel configuration of the running Linux system

```
sudo modprobe configs
sudo zcat /proc/config.gz > kernel.config
cat kernel.config
```

15.5 PWM Fan Control

The following script temporarily runs the fan at maximum speed for a short period (2 seconds). After the temperature drops, it will automatically return to the normal speed:

```
#!/bin/bash

STATE=$1
true ${STATE:=1}

find_pwm_fan() {
    for d in /sys/class/thermal/cooling_device*; do
        if grep pwm-fan $d/type >/dev/null; then
            PWM_FAN=$d
            break
        fi
    done
}

find_pwm_fan

if [ -n "${PWM_FAN}" ]; then
    MAX_STATE=$(cat ${PWM_FAN}/max_state)
    if [ ${STATE} -gt ${MAX_STATE} ]; then
        STATE=${MAX_STATE}
    fi
fi
```

```
echo $STATE > ${PWM_FAN}/cur_state
echo "Current pwm-fan state: $(cat ${PWM_FAN}/cur_state)"
fi
```

15.6 Clear the Automatically Generated MAC Address

- Note: The following method only applies to Linux systems that use GPT partition tables. Since it performs raw writes to the storage device, please back up your data before proceeding.
- Background: When you clone a TF card using the dd command and run it on another development board, you may find that both boards end up using the same MAC address. To resolve this issue, you can use the following command to clear the stored MAC address, allowing the system to generate a new MAC address during the next boot, thereby avoiding duplicate MAC addresses:

```
dd if=/dev/zero of=/dev/mmcblk0 bs=512 seek=7168 count=1
```

In the command above, mmcblk0 refers to the TF card device name.

If the storage device is eMMC, it should be changed to mmcblk2.

If the storage device is UFS, it should be changed to sda.

15.7 Scheduled Power On/Off

Usage example, e.g. configure the board to automatically power on 120 seconds after shutdown:

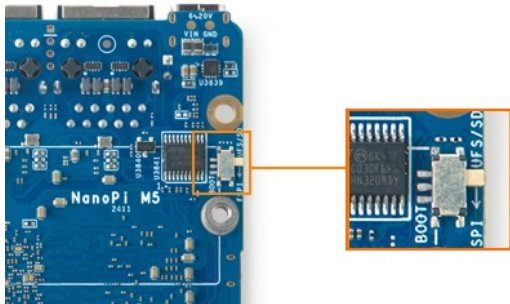
```
echo +120 > /sys/class/rtc/rtc0/wakealarm
poweroff
```

16 Unbricking Method

If the ROM is not installed correctly, causing the development board to become bricked, and you might not have the opportunity to reinstall the ROM via an SD card, you need to enter Maskrom mode to unbrick it by erasing the storage device.

16.1 Preparation

Set the BOOT switch on the back of the PCB to the 'UFS/SD' position:



Note:

- If the BOOT switch is not set to the "UFS/SD" position, the UFS storage may not be recognized.
- The SPINOR storage is always recognized, regardless of the BOOT switch position.

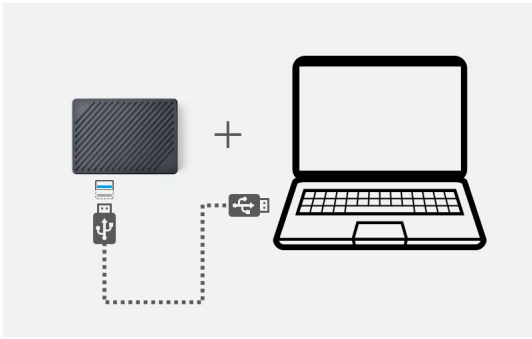
16.2 Windows Users

16.2.1 Download Required Files

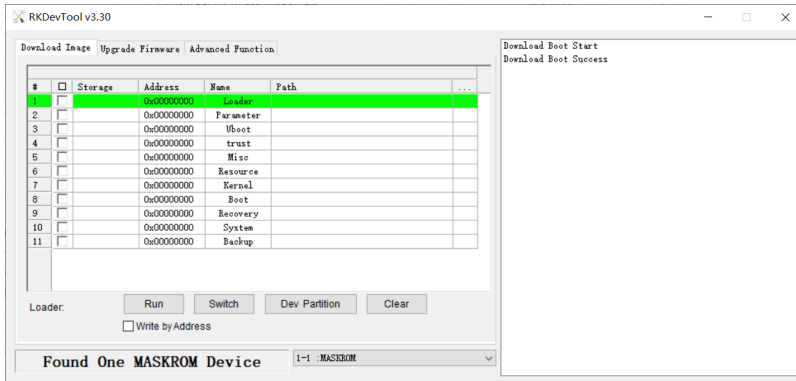
- **Get the necessary tools:** Visit here (<https://dl.friendlyelec.com/NanoPi-M5>), find **RKDevTool_v3.37_for_window.zip** and **DriverAssitant_v5.12.zip** in the **05_Tools** directory, and download them to your local machine.
- **Install Rockchip USB driver and RKDevTool:** Extract **DriverAssitant_v5.12.zip** to install the Rockchip USB driver, and extract **RKDevTool_v3.37_for_window.zip** to obtain the Rockchip flashing tool **RKDevTool**.
- **Get the loader:** Visit here (<http://112.124.9.243/dvdfiles/>), enter the tools directory corresponding to your CPU model, and download **MiniLoaderAll.bin**.

16.2.2 Enter Maskrom Mode to Erase the Storage Device

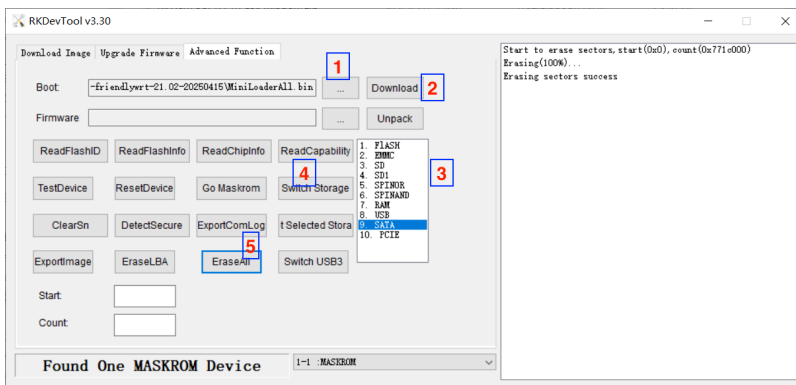
- Remove the SD card, USB device, and other peripherals from the development board
- Start **RKDevTool** on your computer.
- Press and hold the "Mask" key, power on the board. After the status LED has been on for at least 3 seconds, release the Mask key
- Use a USB A-to-A cable, connect NanoPi-M5 to a PC as follows



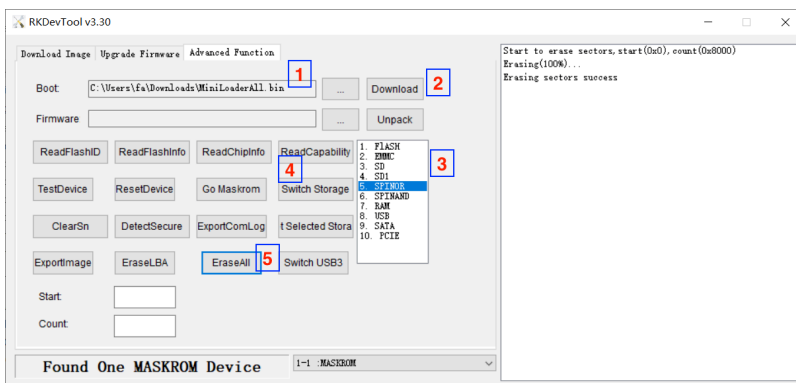
- You will see **Found One MASKROM Device** displayed at the bottom of the **RKDevTool** interface, as shown below:



- Click the **Advanced Function** tab in the **RKDevTool** interface.
- In the **Boot** text box, select **MiniLoaderAll.bin**, then click the **Download** button.
- Select **SATA**, click **Switch Storage**, then click the **EraseAll** button to erase the UFS.



- (Optional): If your NanoPi-M5 has SPI Nor Flash, select **SPINOR**, click **Switch Storage**, then click the **EraseAll** button to erase the SPI Nor Flash.



- At this point, NanoPi-M5 is restored to its initial state and can be normally booted via SD card or UFS.

16.3 Linux Users

16.3.1 Download the Required Files

- Get the necessary tools:** Visit here (<https://dl.friendlyelec.com/NanoPi-M5>) and find **upgrade_tool_v2.30_for_linux.tgz** in the **05_Tools** directory and download it locally.

- **Get the loader:** Visit here (<http://112.124.9.243/dvdfiles/>), enter the tools directory corresponding to your CPU model, and download **MiniLoaderAll.bin**.

16.3.2 Installation for upgrade_tool

Using the following commands:

```
tar xzf upgrade_tool_v2.30_for_linux.tgz
cd upgrade_tool_v2.30_for_linux
sudo cp upgrade_tool /usr/local/sbin/
sudo chmod 755 /usr/local/sbin/upgrade_tool
```

16.3.3 Enter Maskrom Mode to Erase the Storage Device

- Connect NanoPi-M5 to the computer using a USB data cable.
- Disconnect the power from NanoPi-M5, hold down the **MASK** button, connect the power, and release the button after 4 seconds.
- Check the connection with the following command:

```
upgrade_tool LD
```

A result similar to "DevNo=1 Vid=0x2207,Pid=0x350b,LocationID=13 Mode=Maskrom SerialNo=" indicates that the device has been detected.

- Erase the UFS with the following command:

```
upgrade_tool EF MiniLoaderAll.bin
```

- **(Optional):** If your NanoPi-M5 has SPI Nor Flash, erase the SPI Nor Flash with the following commands:

```
upgrade_tool DB MiniLoaderAll.bin
upgrade_tool SSD # Select 5, SPINOR
dd if=/dev/zero of=zero.img bs=1M count=16 # For 16M NOR FLASH
upgrade_tool WL 0 zero.img
```

- At this point, NanoPi-M5 has been restored to its initial state and can boot the system normally via SD card or UFS.

16.4 Mac Users

Our tests found that upgrade_tool_v2.25 does not work properly on macOS. Therefore, we recommend using Windows or Linux unless an updated version of upgrade_tool becomes available.

17 How to use CAM415 MIPI Camera

- Refer to:
 - Camera
 - DTS files

18 Link to Rockchip Resources

19 Schematic, PCB CAD File

- Schematic: NanoPi_M5_LP4X_2411_SCH.PDF (https://wiki.friendlyelec.com/wiki/images/1/10/NanoPi_M5_LP4X_2411_SCH.PDF)
- NanoPi_M5_LP5_2411_SCH.pdf (https://wiki.friendlyelec.com/wiki/images/9/97/NanoPi_M5_LP5_2411_SCH.pdf)
- PCB CAD File: NanoPi_M5_2411_DXF.zip (https://wiki.friendlyelec.com/wiki/images/e/e1/NanoPi_M5_2411_DXF.zip)
- RK3576 Datasheet: Rockchip_RK3576_Datasheet_V1.5-20241216.pdf (https://wiki.friendlyelec.com/wiki/images/b/bf/Rockchip_RK3576_Datasheet_V1.5-20241216.pdf)

20 Update Logs

20.1 2026-06-03

20.1.1 eFlasher (<https://wiki.friendlyelec.com/wiki/index.php/EFlasher>)

- improved the backup and restore functionality
- Added boot logo generation and customization
- Added kernel command-line parameter editing

20.1.2 FriendlyWrt

- Updated FriendlyWrt to OpenWrt 25.12.4

20.2 2026-04-21

20.2.1 FriendlyWrt

- Updated FriendlyWrt to OpenWrt 25.12

- Updated eMMC-Tools web-based flashing tool under FriendlyWrt, improved stability and support for more formats
- Enabled built-in fq_codel queue scheduler in kernel to improve network latency

20.2.2 Ramdisk

- Fixed the issue of factory reset not working on UFS devices
- Optimized the disk repair process

20.2.3 Debian 13(Trixie) GNOME Wayland Desktop

- Fixed duplicate kernel log output on serial port
- Updated Chromium web browser to new version 144.0.7559.133-1

20.3 2026-03-02

20.3.1 Debian 13(Trixie) GNOME Desktop

- Update Chromium web browser to the new version 144.0.7559.133-1
- Optimize boot speed and kernel log output for Debian 13 (Trixie) Desktop
- Add CAM415 camera support

20.3.2 Debian11 & Ubuntu20

- Fix the issue of CAM415 camera 3A not working on Debian 11 and Ubuntu 20

20.4 2026-02-08

- Added desktop system: Debian 13 (Trixie) GNOME Wayland Desktop
- Added desktop system: Ubuntu 24 (Noble) LXQt X11 Desktop
- Added Linux gaming system: ROCKNIX

20.5 2026-01-12

20.5.1 OpenMediaVault

- Update to 8.0.6-1
- Update to Debian13

20.5.2 Debian Core

- Update to Debian13

20.6 2025-12-22

20.6.1 Android 14 / TV

- Sync the factory SDK update to **rkr8**
- Fix an issue where one RK 3A ISP stream did not work when two MIPI cameras were connected

20.6.2 Linux

- Update the vendor kernel to version 6.1.141
- Fix an issue where one RK 3A ISP stream did not work when two MIPI cameras were connected
- Update the Chromium web browser to version 143.0.7499.40 (Debian/Ubuntu only)

20.6.3 FriendlyWrt

- Update FriendlyWrt to version 24.10.4

20.7 2025-10-14

20.7.1 Android 14/TV

- Fix issue where R76S only detects one PCIe lane

20.8 2025-10-10

20.8.1 Android 14 / TV

- Sync the factory SDK update to **rkr7.2**
- Fix failure to auto-expand partitions

20.8.2 Eflasher

- Allow GUI info to be set in config file

20.9 2025-09-05

20.9.1 Android 14/TV

- Updated SDK to RKR7.1 and kernel to 6.1.141
- Fixed black screen at left side in Settings
- For AndroidTV, improved DSI panel support and fixed no-boot issue in portrait mode

20.10 2025-08-11

20.10.1 Ubuntu / Debian / FriendlyCore

- Updated and pre-installed the camera support package camera_engine_rkaiq to the new version 6.30.4
- Added support for the CAM415 camera (How to test (<https://wiki.friendlyelec.com/wiki/index.php/Camera>))
- Updated the ramdisk to fix known issues
- Optimized the configuration of mpv

20.11 2025-08-07

20.11.1 Linux

- Synchronized upstream kernel update to version 6.1.118
- Add support for the HD702M LCD screen (LCD-HD702M (<https://wiki.friendlyelec.com/wiki/index.php/LCD-HD702M>))
- Add GPIO pin labels in the DTS so that the application layer can conveniently access GPIOs via libgpiod (GPIO Programming: Using the libgpiod Library (https://wiki.friendlyelec.com/wiki/index.php/GPIO_Programming:_Using_the_libgpiod_Library))

20.11.2 Android 14

- Sync with the upstream kernel update to version 6.1.118
- Add support for the HD702M LCD screen (LCD-HD702M (<https://wiki.friendlyelec.com/wiki/index.php/LCD-HD702M>))
- Add support for the MIPI camera sensor IMX415
- Enable wireless ADB functionality once Ethernet is connected
- Update the boot logo to match the Android boot animation
- Fix the issue of high-frequency logging that occurs during camera preview

20.12 2025-07-08

20.12.1 FriendlyWrt

- Fixed PWM fan control issue (using the pwm-fan driver module)

20.13 2025-06-30

20.13.1 FriendlyWrt

- Updated FriendlyWrt to version 24.10.2
- Updated network-related kernel configuration

20.14 2025-06-13

20.14.1 Linux Kernel 6.1.y

- Improved sleep/wake support
 - Fixed network unavailability after sleep/wake with rtl8211f
 - Enhanced USB Bluetooth sleep/wake support
- Optimized Bluetooth support
 - Added Bluetooth support for rtl8851bu
 - Re-enabled MediaTek module Bluetooth support, except for the mt7921u 0e8d:7961 module (CF-953AX), which has been disabled due to malfunction during testing

20.15 2025-04-08

Initial Release

Retrieved from "https://wiki.friendlyelec.com/wiki/index.php?title=NanoPi_M5&oldid=28302"

Category: Pages with broken file links

-
- This page was last modified on 28 July 2026, at 06:13.